(α, β)-Core Query on Structured Encrypted Bipartite Graph

Yulin Wu, Lanxiang Chen, *Senior Member, IEEE*, Yi Mu, and Robert H. Deng, *Fellow, IEEE*

***Abstract*—An essential task in bipartite graph analysis is computing the (α, β)-core based on specified α and β values. Existing methods often sacrifice user data security to enhance efficiency and accuracy, either by traversing the entire bipartite graph or pre-processing multiple (α, β) combinations. While recent efforts have introduced privacy-preserving schemes for efficient (α, β)-core queries, they primarily focus on node cohesion, overlooking valuable information embedded in both nodes and edges. In practical applications, analyzing both node and edge attributes is crucial for deriving meaningful insights. In this paper, we propose a novel approach for privacy-preserving (α, β)-core queries on bipartite graphs: (1) Efficient core computation: Instead of pre-processing all (α, β) combinations, we construct index tables and dynamically generate the adjacency matrix in real-time for (α, β)-core computation. (2) Privacy-preserving query: We design a structured encryption scheme for bipartite graphs, integrating a secure comparison protocol and a secure minimum value protocol based on symmetric homomorphic encryption to ensure privacy protection. (3) Optimized graph traversal: To avoid traversing the entire bipartite graph, we employ a bipartite graph coloring method, leveraging partition index tables and degree tables to efficiently identify query vertices that meet the specified degree constraints. (4) Extended query capabilities: Our (α, β)-core query method supports privacy-preserving (α, β)-weighted community ((α, β)-WC) and (α, β)-attribute weighted community ((α, β)-AWC) queries. (5) Performance and security evaluation: Experimental results on real datasets show that the (α, β)-WC query on ciphertext incurs an overhead of 0.49X to 4.89X compared to its plaintext counterpart, while the (α, β)-AWC query incurs an overhead of 0.74X to 4.52X. Additionally, rigorous security analysis validates the robustness of our proposed scheme.**

***Index Terms*—Bipartite graphs, structured encryption, (α, β)-core, symmetric homomorphic encryption, privacy preserving.**

Received 4 August 2025; revised 16 April 2026; accepted 14 July 2026. Date of publication 20 July 2026; date of current version 1 September 2026. This work was supported in part by the Natural Science Foundation of Fujian Province in China under Grant 2024J01068, in part by National Natural Science Foundation of China under Grant 62072105, and in part by the Science Foundation of Zhejiang Sci-Tech University (ZSTU) under Grant 26222226-Y. *(Corresponding author: Lanxiang Chen.)*

Yulin Wu is with the School of Information Science and Engineering, Zhejiang Sci-Tech University, Hangzhou 310018, China, and also with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China.

Lanxiang Chen is with the School of Information Science and Engineering, Zhejiang Sci-Tech University, Hangzhou 310018, China, also with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China, and also with the Faculty of Data Science, City University of Macau, Macao 999078, China (e-mail: lxiangchen@gmail.com).

Yi Mu is with the Faculty of Data Science, City University of Macau, Macao 999078, China (e-mail: ymu.ieee@gmail.com).

Robert H. Deng is with the School of Computing & Information Systems, Singapore Management University, Singapore 188065 (e-mail: robertdeng@smu.edu.sg).

Digital Object Identifier 10.1109/TDSC.2026.3714883

I. Introduction

BIPARTITE graphs are widely used to represent relationships between different types of entities in various real-world scenarios. For instance, they model associations between authors and papers [1], customers and products [2], and patients and diseases [3]. In these practical application networks, inherent community structures emerge, leading to the adoption of various cohesive subgraph models, such as the (α, β)-core [4], bitruss [5], and bicliques [6], to effectively capture and analyze community relationships within bipartite graphs. Leveraging these models, community search in bipartite graphs aims to identify tightly connected subgraphs that satisfy specific structural cohesion criteria. Among them, (α, β)-core queries serve as a powerful subgraph modeling technique with broad applications in fault-tolerant group recommendation [7] and community discovery [8], [9].

By identifying densely connected subgraphs, (α, β)-core queries uncover latent structural associations among entities, thereby enhancing user experience and enabling more in-depth analytical insights. In particular, these queries play a central role in social network analysis, as they facilitate the identification of communities exhibiting strong internal connectivity and structural cohesion. As a result, (α, β)-core analysis has become increasingly valuable in practical applications such as social media platforms, targeted advertising, and personalized recommendation systems, where high accuracy and sustained user engagement are critical.

The rapid expansion of large-scale bipartite graph datasets poses substantial challenges for data storage, processing, and query execution. To address these challenges, both enterprises and individual users are increasingly inclined to outsource data management and query services to third-party providers. Although this paradigm significantly improves scalability and operational efficiency, it also introduces serious security and privacy risks, particularly with respect to the potential leakage of sensitive personal information.

In many real-world scenarios, both node attributes and edge weights encapsulate highly sensitive data. For example, in social network-based community discovery and group recommendation, node attributes may encode private user profiles or personal interests, while edge weights often reflect interaction strength or similarity between users. Unauthorized disclosure of such information during subgraph analysis may reveal private relationships and individual preferences. Likewise, in personalized recommendation and targeted advertising systems, users and items naturally form a weighted bipartite graph, where node

attributes (e.g., user preferences or item features) and edge weights (e.g., click frequencies or relevance scores) are equally sensitive. Ensuring the confidentiality of these elements during query processing is therefore essential for preserving user privacy.

Consequently, advancing (α, β)-core query techniques requires not only scalable and efficient computation over large bipartite graphs, but also robust privacy-preserving mechanisms that protect sensitive attributes and structural information throughout the entire query.

Recently, Guan et al. [10] proposed an efficient and privacy-preserving (α, β)-core query scheme for bipartite graphs in the cloud. Their approach constructs two index tables for all possible combinations of α and β to facilitate efficient query execution. Additionally, they employ symmetric homomorphic encryption and encrypted queues to ensure data security and preserve the privacy of query results. Several other schemes [11], [12], [13] have adopted k-anonymity techniques to protect graph data privacy. However, such approaches are inherently unsuitable for supporting exact (α, β)-core queries and fail to protect the privacy of query requests. Meanwhile, alternative solutions relying on homomorphic encryption or secure computation protocols [14], [15] are not directly applicable to (α, β)-core queries in bipartite graph settings.

Moreover, existing privacy-preserving (α, β)-core query schemes primarily concentrate on node structural cohesion, overlooking node attributes. This limitation may significantly degrade query accuracy in application scenarios where attribute information plays a critical role.

Consequently, a fundamental challenge remains open: how to design a privacy-preserving (α, β)-core query framework that simultaneously captures both node structural cohesion and attribute homogeneity, while rigorously protecting graph data and query privacy.

Our Contributions: To address the aforementioned challenges, this paper proposes a novel community search query scheme for structured encrypted bipartite graphs. The key contributions are summarized as follows:

1) The (α, β)-core query scheme proposed by Guan et al. [10] requires preprocessing all combinations of α and β, leading to substantial indexing overhead. To overcome this limitation, we propose a new (α, β)-core computation method that dynamically constructs an adjacency matrix in real-time, enabling efficient query execution.
2) We employ structured encryption techniques to secure the graph and design secure comparison and minimum value protocols using symmetric homomorphic encryption. These protocols ensure privacy-preserving (α, β)-core computation.
3) Existing schemes fail to protect node attribute information and edge weight information in bipartite graphs. To address this gap, we introduce keyword index tables for attributes and weight tables for edges. Building on the basic (α, β)-core query, we develop privacy-preserving (α, β)-weighted community ((α, β)-WC) and (α, β)-attribute weighted community ((α, β)-AWC) queries.
4) We conduct a comprehensive evaluation of our scheme using real datasets. Experimental results show that the proposed (α, β)-WC queries incur an additional overhead of 0.49X to 4.89X compared to the plaintext setting, while (α, β)-AWC queries introduce an extra overhead of 0.74X to 4.52X. Furthermore, we validate the security of our scheme through rigorous security analysis.

II. Related Work

In 2010, Chase and Kamara [16] introduced structured encryption (STE), extending symmetric searchable encryption (SSE) to support diverse data types. This advancement leveraged the concept of association, allowing simple encryption schemes to be combined into more complex ones. Structured encryption is particularly well-suited for handling intricate data types, such as social network graphs. Since then, numerous privacy-preserving graph query schemes [17], [18], [19], [20], [21], [22], [23] have been proposed.

Unlike previous works, this paper extends structured encryption to specialized graph types, specifically bipartite graphs, enabling fundamental graph query operations in a more comprehensive manner.

Cohesive subgraph models, including (α, β)-core, bitruss, and bicliques, serve as key techniques for characterizing and analyzing community structures in bipartite graphs. Among these, the (α, β)-core model, inspired by the k-core concept, imposes distinct constraints on vertices in different layers. In 2017, Ding et al. [7] proposed GraphRec, a client-side fault-tolerant group recommendation approach based on (α, β)-core principles. GraphRec was designed to address query latency challenges in large-scale datasets. However, a significant drawback was its need to traverse the entire graph for computing the (α, β)-core, making it impractical for large-scale bipartite graphs.

To improve efficiency, Liu et al. [24] introduced the BiCore-Index in 2019, along with an optimal (α, β)-core computation algorithm. By leveraging the (α, β)-core decomposition structure, this method enhanced query efficiency by indexing and selectively traversing the graph. However, the algorithm's centralized nature required maintaining the entire graph's degree information sequentially, deleting vertices from highest to lowest degree until the graph became empty. This approach was unsuitable for distributed environments where the graph is partitioned and stored across multiple devices.

To address this limitation, Liu et al. [25] proposed a distributed (α, β)-core decomposition algorithm using an n-order double index structure called Bi-indexes. This algorithm enabled (α, β)-core decomposition in distributed settings where graphs are stored across multiple devices.

To further enhance privacy protection for sensitive data, Guan et al. [10] developed an efficient and privacy-preserving (α, β)-core query scheme. Their approach constructed two index tables based on the bipartite graph and employed symmetric homomorphic encryption to establish an encrypted queue, ensuring that the cloud server could not access query requests or results.

However, all these approaches primarily focus on structural cohesion, overlooking the significance of weighted interactions between vertices and their attributes. In real-world community search scenarios, community formation is influenced not only

by vertex connectivity but also by edge weight interactions and vertex attributes. Edge weights may signify the strength or significance of relationships, while vertex attributes contain valuable characteristics. Neglecting these factors can lead to incomplete or inaccurate community search results.

In 2021, Wang et al. [8] explored weighted interactions and vertex attributes in community search, introducing the (α, β)-WC model. Their method utilized index structures and peeling-expansion algorithms to identify (α, β)-WC. However, it focused solely on edge weight information, disregarding vertex attributes, limiting its applicability in personalized recommendation scenarios. In 2022, Li et al. [26] proposed a more comprehensive approach incorporating pruning strategies, exact algorithms, and approximation techniques to enhance attribute cohesion in community structures. This advancement improved applications such as personalized recommendations, fraud detection, and temporal organization. In 2023, Xu et al. [27] introduced an attribute-aware (α, β)-community search algorithm. Their method generated candidate keyword sets and validated community existence based on attribute information, facilitating efficient (α, β)-community search on attribute bipartite graphs.

Overall, research has made significant progress in incorporating weight values and vertex attributes in community search problems, proposing various algorithms to enhance accuracy and applicability. However, most bipartite graph community search methods operate in plaintext settings, overlooking data security. Although Guan et al. [10] proposed a privacy-preserving (α, β)-core query scheme that enumerates all node-based α and β combinations, it does not consider attribute information in real-world applications. Therefore, developing a privacy-preserving community search scheme for bipartite graphs that incorporates both edge weight and vertex attribute information remains an essential research direction.

III. Problem Formulation

A. Notations

The symbols employed in this paper are outlined in Table I. The pseudo-random functions (PRFs) and pseudo-random permutations (PRPs) mentioned in the paper are formally defined as follows.

1) $f : \{0,1\}^{\eta} \times \{0,1\}^{*} \to \{0,1\}^{\lambda}$.
2) $H_{k_1/k_2} : \{0,1\}^{\lambda} \times \{0,1\}^{*} \to \{0,1\}^{*}$.
3) $F_{k_3} : \{0,1\}^{\lambda} \times \{0,1\}^{*} \to \{0,1\}^{\log(n)+\lambda}$.
4) $\pi : [n] \to [n]$.

Definition 1 (Symmetric-key encryption): A symmetric-key encryption (SKE) scheme consists of three algorithms, denoted as $\Phi = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$:

- $\mathsf{Gen}(1^{\lambda}) \to \mathsf{k}$: Takes input 1^{λ} and outputs a secret key k_4.
- $\mathsf{Enc}(\mathsf{k}_4, \mathsf{m}) \to \mathsf{c}$: A probabilistic algorithm that, given key k_4 and plaintext m, outputs ciphertext c.
- $\mathsf{Dec}(\mathsf{k}_4, \mathsf{c}) \to \mathsf{m}$: A deterministic algorithm that, given key k_4 and ciphertext c, recovers plaintext m.

Definition 2 ((α, β)-core): Given a bipartite graph $G = (U, L, E)$ and integers α and β, a maximal subgraph $\mathcal{R} = (U_1, L_1, E_1) \subseteq G$ is an (α, β)-core if every vertex in U_1 has degree at least α, and every vertex in L_1 has degree at least β, i.e., $\forall u \in U_1, \deg(u, \mathcal{R}) \geq \alpha \wedge \forall v \in L_1, \deg(v, \mathcal{R}) \geq \beta$.

TABLE I
Notations

Symbol	Description
$G(V, E)$	A bipartite graph and $V = (U, L)$.
n	The number of nodes.
$\mathbf{M}$	Data items.
m	Plaintext message.
S	Partition index table.
D_1	Query degree index table.
D_2	Vertex degree index table.
W	The keyword set.
$\bar{W}$	The weight set $\bar{W} = \{\bar{w}_1, \cdots, \bar{w}_n\}$.
$I_{\bar{W}}$	Weight value index table.
λ	Security parameter.
$\mathbf{c}$	The set of ciphertexts $\mathbf{c} = \{c_1, \ldots, c_n\}$.
γ	Secure index set.
τ	Query token.
K	The key set $K = \{k_1, \cdots, k_5\}$.
ϑ	The semi-private data.
t	Query type.
A	The adjacency matrix $A = \{a_{ij}\}$.
$\Lambda(G)$	Info leaked in response to a query.

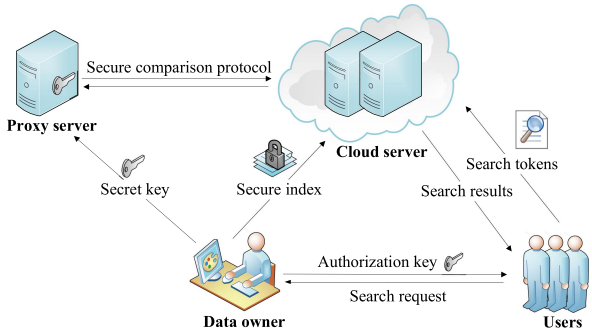


Fig. 1. The system model of STE-BG with four entities: data owner (DO), cloud server (CS), proxy server (PS) and users.

Definition 3 (Bipartite graph weight): For a bipartite graph G, the weight $\bar{W}(G)$ is defined as the minimum edge weight: $\bar{W}(G) = \min_{e \in E(G)} W(e)$.

Definition 4 ((α, β)-weighted community): Given a bipartite graph $G = (U, L, E)$, integers α and β, and a query node q, the goal is to identify a maximal connected subgraph $\mathcal{R}_q = (U_1, L_1, E_1) \subseteq G$ containing q such that: $\forall u \in U_1, \deg(u, \mathcal{R}_q) \geq \alpha \wedge \forall v \in L_1, \deg(v, \mathcal{R}_q) \geq \beta$, and $\mathcal{R}_q$ has the maximum weight value $W(\mathcal{R}_q)$.

Definition 5 ((α, β)-attribute weighted community): Given a bipartite graph G, a query node q, degree constraints α and β, and a set of query attributes (keywords) q_w, the goal is to identify a subgraph $\mathcal{R}_{q_w}$ containing q such that: $\forall u \in U_1, \deg(u, \mathcal{R}_q) \geq \alpha \wedge \forall v \in L_1, \deg(v, \mathcal{R}_q) \geq \beta$, and the attribute set of vertices in L_1 includes q_w, while maximizing the weight value $\bar{W}(\mathcal{R}_q)$.

B. System and Threat Model

The system model involves four entities: the data owner (DO), the cloud server (CS), the proxy server (PS), and authorized users, as illustrated in Fig. 1.

We assume that the cloud server and the proxy server are *non-colluding*. Under this assumption, the following trust relationships are defined.

- *Data Owner (DO):* The DO is fully trusted with respect to key generation and key management. It encrypts the graph data and associated indexes locally before outsourcing them to the cloud. The DO does not trust either the CS or the PS.
- *Cloud Server (CS):* The CS is responsible for storing and processing encrypted data outsourced by the DO. It is assumed to be *honest-but-curious:* while correctly executing the prescribed protocols and collaborating with the proxy server to perform computationally intensive operations, the CS does not have access to any secret keys and may attempt to infer sensitive information from the ciphertexts.
- *Proxy Server (PS):* The PS is also assumed to be *honest-but-curious* and does not collude with the CS. Its role is limited to assisting with computationally intensive operations over encrypted data. The PS does not have access to plaintext data or decryption keys and cannot independently recover sensitive information.
- *Users:* Authorized users are allowed to issue query requests and receive encrypted query results. They do not directly interact with the encrypted index structures stored on the CS and cannot access secret keys beyond those explicitly granted by the DO.

The security of the proposed system relies on the non-collusion assumption between the CS and the PS. Specifically, neither party alone can learn sensitive information about the outsourced data or user queries beyond what is allowed by the defined leakage. We do not consider attacks in which the CS and PS collude, as such collusion would undermine the separation-of-duty design and compromise system security.

The data owner securely stores a set of encrypted index lists on the cloud server. To facilitate secure and efficient searches, the DO generates a set of secure indexes γ and creates retrieval tokens τ in response to user queries. Upon receiving a valid search token, the CS, together with the PS, executes the prescribed encrypted query protocol and returns the corresponding encrypted results. Finally, authorized users decrypt the ciphertext associated with pointer $\mathbf{c}[J]$ using the secret key provided by the DO.

*Definition 6 (*STE-BG*):* A privacy-preserving (α, β)-core query scheme over structured encrypted bipartite graphs is defined as $\mathsf{STE\text{-}BG} = (\mathsf{GenKey}, \mathsf{EncData}, \mathsf{Token}, \mathsf{Search}, \mathsf{DecData})$, as detailed below:

- $\mathsf{GenKey}(1^\lambda) \to K$: Given the security parameter λ, the data owner generates a set of secret keys $K = \{k_1, k_2, k_3, k_4, k_5\}$.
- $\mathsf{EncData}(K, G, \mathbf{M}) \to (\gamma, \mathbf{c})$: On input the key set K, a bipartite graph G, and the associated data items $\mathbf{M}$, this algorithm encrypts the graph-related data using PRPs, PRFs, and symmetric encryption techniques, and outputs a secure index structure γ together with the corresponding ciphertexts $\mathbf{c}$.
- $\mathsf{Token}(K, \alpha, \beta, q_u, q_w, t) \to \tau$: Given the key set K and a user query request, the data owner generates a query token τ. Here, α and β denote the degree constraints, q_u represents the queried node, q_w denotes the queried keyword, and t specifies the query type.
- $\mathsf{Search}(\gamma, \tau) \to (A^*, J)$: Upon receiving the query token τ, the cloud server, with assistance from the proxy server, performs secure computations over the encrypted index γ and outputs the encrypted adjacency matrix A^* and a ciphertext pointer J.
- $\mathsf{DecData}(K, A^*, J, \mathbf{c}) \to (\hat{A}, \mathbf{m})$: The user decrypts the ciphertext components J and A^* using the keys k_4 and k_5, recovering the corresponding plaintext information $(\hat{A}, \mathbf{m})$.

C. Security Model

In this paper, we focus on achieving privacy-preserving (α, β)-core queries on bipartite graphs using structured encryption. Consequently, the security of STE-BG can be effectively reduced to the security of structured encryption.

In our system model, we assume that PS and CS are non-colluding entities hosted by independent cloud providers (e.g., Google and Amazon). This implies that an attacker cannot compromise more than one server simultaneously. This assumption is reasonable, as these providers have reputational incentives against collusion, which has been a widely accepted premise in recent works [28], [29]. Additionally, the proxy server (PS) is considered semi-honest, meaning it assists in computation but cannot disclose sensitive data, ensuring the security model remains intact.

The security guarantees of our proposed graph encryption scheme can be summarized as follows: (1) Given an encrypted data structure γ and a set of ciphertext sequences $\mathbf{c}$, no adversary should gain partial information about $\mathbf{m}$. (2) Given a set of tokens $\tau = (\tau_1, \ldots, \tau_l)$ corresponding to adaptively generated queries $\mathbf{q} = (q_1, \ldots, q_l)$, apart from the information revealed by semi-private data $(\vartheta_1, \ldots, \vartheta_l)$, no adversary should infer partial information about $\mathbf{m}$ or $\mathbf{q}$. Achieving these guarantees is challenging, necessitating a formal security analysis that accounts for controlled leakage.

Definition 7 (Query Pattern and Access Pattern): The query pattern (QP) indicates whether specific vertices have been queried. Let $\mathbf{q}$ be a non-empty query set. For any query $q \in \mathbf{q}$, the i-th element of the vector is 1 if $q = q_i$ and 0 otherwise. The access pattern $\mathrm{AP}(\mathbf{q})$ is defined as $\mathrm{AP}(\mathbf{q})[i] = \pi[I']$, where π is a pseudo-random permutation and $I' = \mathsf{Search}(\gamma, \mathbf{q})$. If a data user issues the same query in G, this pattern is exposed to the attacker; however, the actual messages remain secure as each entry in $\mathbf{c}$ is obfuscated and encrypted.

($\mathcal{L}_1$, $\mathcal{L}_2$)-Leakage Functions: Given a bipartite graph G, encryption produces a structure γ and a set of ciphertexts $\mathbf{c}$. These ciphertexts expose metadata such as dataset size, captured by the first leakage function $\mathcal{L}_1(G, \mathbf{M})$. For each query $q \in \mathbf{q}$, the query pattern $\mathrm{QP}(q)$ and access pattern $\mathrm{AP}(q)$ may be inferred from tokens, modeled as $\mathcal{L}_2(\gamma, q)$. These leakage functions primarily arise during query execution.

To assess security, we analyze our scheme under Adaptively Chosen Query Attacks (CQA2), leveraging leakage functions defined in [16].

Definition 8 (CQA2-Security): Given a graph encryption scheme $\Sigma = (\mathsf{GenKey}, \mathsf{EncData}, \mathsf{Token}, \mathsf{Search}, \mathsf{DecData})$, we define two security games:

Real$_{\mathcal{A}}(\lambda)$*:* The challenger executes $\mathsf{GenKey}(1^{\lambda})$ to generate keys K. The adversary $\mathcal{A}$ selects $(G, \mathbf{M})$ and receives $(\gamma, \mathbf{c}) \leftarrow \mathsf{EncData}(K, G, \mathbf{M})$. $\mathcal{A}$ then adaptively issues polynomially many queries $(\alpha, \beta, q_u, q_w, t)$ and obtains query tokens $\tau \leftarrow \mathsf{Token}(K, \mathbf{q})$ from the challenger. Query results $(A^*, J) \leftarrow \mathsf{Search}(\gamma, \tau)$ are then retrieved. Finally, $\mathcal{A}$ outputs a bit $b \in \{0, 1\}$ based on the obtained information.

Ideal$_{\mathcal{A},\mathcal{S}}(\lambda)$*:* The adversary $\mathcal{A}$ selects $(G, \mathbf{M})$. Given $\mathcal{L}_1(G, \mathbf{M})$, the simulator $\mathcal{S}$ generates and sends $(\gamma, \mathbf{c})$ to $\mathcal{A}$. $\mathcal{A}$ then issues adaptive queries $q \in \mathbf{q}$. The simulator $\mathcal{S}$ simulates tokens τ based on $(\mathcal{L}_2(\gamma, q), \vartheta)$ and sends them to $\mathcal{A}$, which subsequently retrieves $(A^*, J) \leftarrow \mathsf{Search}(\gamma, \tau)$. Finally, $\mathcal{A}$ outputs a bit $b \in \{0, 1\}$.

If for all polynomial-time adversaries $\mathcal{A}$, there exists a polynomial-time simulator $\mathcal{S}$ such that:

$$|\Pr[\mathbf{Real}_{\mathcal{A}}(\lambda) = 1] - \Pr[\mathbf{Ideal}_{\mathcal{A},\mathcal{S}}(\lambda) = 1]| \leq \mathsf{negl}(\lambda)$$

then the graph encryption scheme is CQA2-secure, where $\mathrm{negl}(\cdot)$ denotes a negligible function.

D. Quantification of Information Leakage

To evaluate privacy leakage in structured encrypted graph querying schemes, we introduce a *leakage inversion* framework grounded in Shannon entropy from information theory [30]. This framework defines two core concepts: **reconstruction space** and **leakage inversion entropy**, which together characterize an adversary's ability to infer the original graph structure from observed query results.

Reconstruction Space: We define the reconstruction space $\mathsf{RS}(G)$ as the set of all plaintext graphs that yield the same observable leakage Λ as the true graph G:

$$\mathsf{RS}(G) = \{G' \mid \Lambda(G') = \Lambda(G)\}.$$

From the adversary's perspective, each $G' \in \mathsf{RS}(G)$ is a plausible candidate for the true underlying graph G.

Leakage Inversion Entropy: To quantify the adversary's uncertainty about G, we adopt Shannon entropy [30] as a metric. Accordingly, the leakage inversion entropy is defined as:

$$\mathrm{LeakInvH}(G, \Lambda) = -\sum_{x \in \mathsf{RS}(G)} P(x) \log P(x) \tag{1}$$

where $P(x)$ denotes the adversary's belief (probability distribution) over candidate graphs. In the absence of auxiliary information, we assume a uniform prior over $\mathsf{RS}(G)$, leading to:

$$\mathrm{LeakInvH}(G, \Lambda) = \log |\mathsf{RS}(G)| \tag{2}$$

Thus, the leakage inversion entropy LeakInvH captures the number of bits of uncertainty, effectively quantifying the adversary's difficulty in reconstructing the original graph structure from the leaked information.

IV. Ciphertext Computation Protocol

A. Basic Knowledge

In this paper, we utilize the symmetric additive homomorphic encryption (SAHE) scheme proposed by Savvides et al. [31] to perform ciphertext operations. This scheme is defined over an Abelian additive group $\mathbb{Z}_N$ of order $N > 1$. Let $\varphi : \{0,1\}^{\lambda} \times \{0,1\}^{\lambda} \to \mathbb{Z}_N$ be a PRF mapping strings of length λ to elements in $\mathbb{Z}_N$. The SAHE scheme consists of a triplet $\Pi = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$, defined as follows:

- $\mathsf{Gen}(1^{\lambda}) \to k_5$: The key generation algorithm takes a security parameter 1^{λ} as input and outputs a symmetric key $k_5 \in \{0,1\}^{\lambda}$.
- $\mathsf{Enc}(k_5, m) \to c$: Given the key k_5, a message $m \in \mathbb{Z}_N$, and a randomly chosen value $r \in \{0,1\}^{\lambda}$, the encryption algorithm outputs the ciphertext $c := \langle \mu, l_p, l_n \rangle = \langle (m + \varphi_{k_5}(r)) \bmod N, [r], \varnothing \rangle$. Here, l_p and l_n are identifier lists (**id**s), where l_p is used for additive masking and l_n for subtractive masking.
- $\mathsf{Dec}(k_5, c) \to m$: Given the key k_5 and the ciphertext $c = \langle \mu, l_p, l_n \rangle$, the decryption algorithm recovers the plaintext $m := (\mu - \sum_{r_1 \in l_p} \varphi_{k_5}(r_1) + \sum_{r_2 \in l_n} \varphi_{k_5}(r_2)) \bmod N$.

The scheme Π retains additive homomorphic properties and ensures security under chosen plaintext attacks (CPA). Let $c_1 = \langle \mu_1, l_{p_1}, l_{n_1} \rangle$, $c_2 = \langle \mu_2, l_{p_2}, l_{n_2} \rangle$, and m be plaintext values. The supported homomorphic operations are:

- *Addition of two ciphertexts:*

$$\mathsf{add}(c_1, c_2) := \langle (\mu_1 + \mu_2) \bmod N, l_{p_1} \cup l_{p_2}, l_{n_1} \cup l_{n_2} \rangle$$

  where $\cup$ denotes list concatenation.
- *Addition of a ciphertext and a plaintext:*

$$\mathsf{adp}(c, m) := \langle (\mu + m) \bmod N, l_p, l_n \rangle$$

- *Multiplication of a ciphertext and a plaintext:*

$$\mathsf{mlp}(c, m) := \langle (\mu \times m) \bmod N, l_1, l_2 \rangle$$

  where:

$$l_1, l_2 = \begin{cases} l_n^{|m|}, l_p^{|m|}, & \text{if } m < 0 \\ \varnothing, \varnothing, & \text{if } m = 0 \\ l_p^{|m|}, l_n^{|m|}, & \text{if } m > 0 \end{cases}$$

  Here, $|m|$ represents the absolute value of m, and $l^{|m|}$ denotes concatenating l with itself $|m| - 1$ times. For example, if $l = [r_1, r_2]$ and $m = 3$, then $l^{|m|} = [r_1, r_2, r_1, r_2, r_1, r_2]$.
- *Negation (inverse) of a ciphertext:*

$$\mathsf{neg}(c) := \mathsf{mlp}(c, -1) = \langle -\mu \bmod N, l_n, l_p \rangle$$

- *Subtraction of two ciphertexts:*

$$\begin{aligned} \mathsf{sub}(c_1, c_2) &:= \mathsf{add}(c_1, \mathsf{neg}(c_2)) \\ &= \langle (\mu_1 - \mu_2) \bmod N, l_{p_1} \cup l_{n_2}, l_{n_1} \cup l_{p_2} \rangle \end{aligned}$$

B. Secure Comparison Protocol

The secure comparison protocol (SCP) enables ciphertext comparisons while preserving privacy.

To compare two ciphertexts c_1 and c_2, a random bit $b \in \{0, 1\}$ is generated. The cloud server (CS) computes $E(\tilde{x})$ using (3) and sends it to the proxy server (PS). Here, r_1 and r_2 are random integers satisfying $0 < r_2 < r_1$. The PS decrypts $E(\tilde{x})$ to obtain the plaintext $\tilde{x}$ and compares it with 0. If $\tilde{x} \geq 0$, then $\sigma = 1$, otherwise $\sigma = 0$. The PS then returns either $\Pi.\mathsf{Enc}(k_5, 1)$ or $\Pi.\mathsf{Enc}(k_5, 0)$ based on σ. Finally, the CS receives the result set R from the PS and compares b with σ. If $b = \sigma$, then $c_1 \geq c_2$; otherwise, $c_1 < c_2$.

$$E(\tilde{x}) = \begin{cases} r_1 \cdot (c_1 - c_2) + r_2, & b = 1 \\ r_1 \cdot (c_2 - c_1) - r_2, & b = 0 \end{cases} \tag{3}$$

The detailed procedure is given in Algorithm 1. Due to space constraints, the security proof is referred to [31]. We give the correctness proof as below.

Correctness Proof: Assume $m_1 < m_2$ and let $0 < r_2 < r_1$.

- If $b = 1$, define $m = m_1 - m_2 < 0$. Then
$$\tilde{x} = r_1 m + r_2 < 0.$$
- If $b = 0$, define $m = m_2 - m_1 > 0$. Then
$$\tilde{x} = r_1 m - r_2 > 0.$$

After receiving $\tilde{x}$, the processing server (PS) outputs

$$\sigma = \begin{cases} 1, & \tilde{x} \geq 0, \\ 0, & \tilde{x} < 0. \end{cases}$$

The client server (CS) then compares σ with b. If $b \neq \sigma$, it concludes $m_1 < m_2$; otherwise, $m_1 \geq m_2$. Hence, the protocol correctly determines the ordering of m_1 and m_2 for any $b \in \{0, 1\}$.

Using the homomorphic property of SAHE, the CS computes $E(\tilde{x})$ and sends it to PS. The PS decrypts to obtain $\tilde{x}$ and returns the sign bit σ as defined above. The CS infers the comparison result from (b, σ). □

Example: Consider the comparison of $E(6)$ and $E(4)$. Let $r_1 = 6$, $r_2 = 2$, and $b = 1$. Since $b = 1$,

$$E(\tilde{x}) = r_1 \cdot (E(6) - E(4)) + r_2 = E(14).$$

The PS decrypts using key k_5:

$$\tilde{x} = \Pi.\mathsf{Dec}(k_5, E(\tilde{x})) = 14.$$

Because $\tilde{x} \geq 0$, PS returns $\sigma = 1$. Since $\sigma = b$, the CS concludes $6 \geq 4$.

C. Secure Minimum Computation Protocol

The secure minimum computation protocol (SMCP) enables the evaluation of the minimum among encrypted inputs without revealing their plaintext values. The protocol is executed jointly by two non-colluding entities, the cloud server (CS), the proxy server (PS), and outputs

$$E(\min\{m_1, m_2, \ldots, m_n\}).$$

0–1 Cipher Encoding: Let $m_1, \ldots, m_n \in X = \{x_1, x_2, \ldots, x_y\}$, where X is a totally ordered domain with $x_1 < x_2 < \cdots < x_y$. Each m_i is represented as a y-dimensional encrypted indicator vector $\tilde{\mathbf{c}}_i = (c_{i1}, \ldots, c_{iy})$ defined by

$$c_{ij} = \begin{cases} E(1), & x_j \leq m_i, \\ E(0), & x_j > m_i, \end{cases} \quad i \in [1, n],\ j \in [1, y]. \tag{4}$$

For each coordinate j, CS homomorphically aggregates all ciphertexts:

$$z_j = \sum_{i=1}^{n} c_{ij}, \qquad \mathbf{z} = (z_1, \ldots, z_y). \tag{5}$$

Hence, z_j encrypts the number of inputs satisfying $m_i \geq x_j$.

A secure ciphertext comparison protocol (denoted by $[[\cdot]]$) is then applied component-wise to determine whether z_j equals the maximum count of $E(1)$, defined as mc. The recombined ciphertext is

$$z'_j = [[z_j, E(\mathsf{mc})]] = \begin{cases} E(1), & \text{if } z_j \text{ encrypts } \mathsf{mc}, \\ E(0), & \text{otherwise.} \end{cases} \tag{6}$$

Finally, the encrypted index of the minimum value is obtained as

$$c_{\mathsf{min}} = \sum_{j=1}^{y} z'_j. \tag{7}$$

Decrypting c_{min} reveals the index of $\min\{m_1, \ldots, m_n\}$ in X, while no intermediate plaintexts are exposed. The full protocol is given in Algorithm 2.

Correctness Proof: Let $m_{\min} = \min\{m_1, \ldots, m_n\} = x_k$. For any $j \leq k$, all $m_i \geq x_j$, hence z_j encrypts the maximum count of $E(1)$, i.e. mc, and then $z'_j = E(1)$. For any $j > k$, at least one $m_i < x_j$, thus $z_j < n$ and $z'_j = E(0)$. Therefore,

$$c_{\mathsf{min}} = \sum_{j=1}^{y} z'_j = k,$$

which is exactly the index of $m_{\min}$ in X. □

Algorithm 1: $\mathsf{SCP}(c_1, c_2) \to R$.

Input: Two ciphertexts c_1 and c_2.
Output: R.
1 : CS**:**
(a) Randomly generates a bit $b \in \{0, 1\}$ and two random integers r_1, r_2 satisfying $0 < r_2 < r_1$;
(b) If $b = 1$, compute $E(\tilde{x}) \leftarrow \mathsf{adp}(\mathsf{mlp}(\mathsf{sub}(c_1, c_2), r_1), r_2)$, otherwise $E(\tilde{x}) \leftarrow \mathsf{adp}(\mathsf{mlp}(\mathsf{sub}(c_2, c_1), r_1), -1 \cdot r_2)$;
(c) Send $E(\tilde{x})$ to PS;
2 : PS**:**
(a) Receive $E(\tilde{x})$ and get $\tilde{x} \leftarrow \Pi.\mathsf{Dec}(k_5, E(\tilde{x}))$;
(b) If $\tilde{x} \geq 0$, $R \leftarrow (\sigma = 1, \Pi.\mathsf{Enc}(k_5, 1), \Pi.\mathsf{Enc}(k_5, 0))$, otherwise $R \leftarrow (\sigma = 0, \Pi.\mathsf{Enc}(k_5, 1), \Pi.\mathsf{Enc}(k_5, 0))$;
(c) Send R to CS;
3 : CS**:**
(a) Receive R from PS;
(b) If $b = \sigma$, $c_1 \geq c_2$, otherwise $c_1 < c_2$.

Algorithm 2: $\mathsf{SMCP}(\{\tilde{\mathbf{c}}_i\}_{i=1}^{n}) \rightarrow c_{\min}$.

Input: Encrypted indicator vectors $\tilde{\mathbf{c}}_i = (c_{i1}, \ldots, c_{iy})$ for $i \in [1, n]$.
Output: Ciphertext $c_{\min} = E(j^\star)$ where $x_{j^\star} = \min\{m_1, \ldots, m_n\}$.

// Step 1: Homomorphic aggregation at CS
for $j = 1$ **to** y **do**
  $z_j \leftarrow c_{1j}$;
  for $i = 2$ **to** n **do**
    $z_j \leftarrow \mathsf{add}(z_j, c_{ij})$;
$\mathbf{z} \leftarrow (z_1, \ldots, z_y)$;
// Step 2: Determine encrypted maximum count mc
$c_{\mathsf{mc}} \leftarrow E(\mathsf{mc})$;
// Step 3: Secure comparison protocol between CS and PS
for $j = 1$ **to** y **do**
  $z'_j \leftarrow [[\, z_j, c_{\mathsf{mc}} \,]]$;
$\mathbf{z}' \leftarrow (z'_1, \ldots, z'_y)$;
// Step 4: Recover encrypted index of minimum
$c_{\min} \leftarrow z'_1$;
for $j = 2$ **to** y **do**
  $c_{\min} \leftarrow \mathsf{add}(c_{\min}, z'_j)$;
return $c_{\min}$;

Example: Let $X = \{1, 2, 3, 4, 5\}$ and $(m_1, m_2, m_3) = (2, 3, 5)$. Using additive homomorphism $E(a + b) = E(a) + E(b)$:

$$\tilde{\mathbf{c}}_1 = (E(1), E(1), E(0), E(0), E(0)),$$
$$\tilde{\mathbf{c}}_2 = (E(1), E(1), E(1), E(0), E(0)),$$
$$\tilde{\mathbf{c}}_3 = (E(1), E(1), E(1), E(1), E(1)).$$

Column-wise aggregation yields

$$\mathbf{z} = (E(3), E(3), E(2), E(1), E(1)).$$

Only z_1 and z_2 encrypt $\mathsf{mc} = 3$, hence

$$\mathbf{z}' = (E(1), E(1), E(0), E(0), E(0)).$$

Thus,

$$c_{\min} = \sum_{j=1}^{5} z'_j = E(2),$$

indicating that the minimum value corresponds to index 2 in X, i.e., $\min\{2, 3, 5\} = 2$.

V. Detailed Index Scheme

A. Index Construction

Basic Index: To reduce the traversal overhead in the bipartite graph G when computing the (α, β)-core, we construct three index tables: the query degree index table D_1, the vertex degree index table D_2, and the partition index table S. As an example, we refer to the illustrated bipartite graph in Fig. 2(a).

- *Query Degree Index Table (D_1):* This table stores vertices whose degrees are greater than or equal to a specified value α (β), allowing efficient retrieval of vertices that satisfy the given constraints. As constraints become stricter, the matrix size decreases accordingly. To construct D_1, consider the example in Fig. 2(b), where the maximum degree of upper-layer vertices is 4. We construct a dictionary for degrees $\{1, 2, 3, 4\}$. For each $i \in \{1, 2, 3, 4\}$, the index key is defined as $i||t_{j\in\{1,2\}}$, and the corresponding index values are stored for all degrees greater than or equal to i. This serves as an entry point for D_2. For example, $D_1(1||t_1) = \{D_2(1||t_1), D_2(2||t_1), D_2(3||t_1), D_2(4||t_1)\}$. Here, t_1 represents the type of the upper-layer vertex, while t_2 represents the type of the lower-layer vertex.
- *Vertex Degree Index Table (D_2):* This table stores vertices with degrees exactly equal to specified values α (β). It maintains mappings between vertex degrees and their corresponding vertex sets.
  As illustrated in Fig. 2(c), consider the case where the degree of upper-layer vertices is 1. The set of vertices satisfying this degree constraint is $\{u_6, u_7\}$, leading to the index entry $D_2(1||t_1) = \{u_6, u_7\}$. Similarly, for lower-layer vertices with degree 2, the set is $\{v_1, v_2, v_6, v_7\}$, and the corresponding index is $D_2(2||t_2) = \{v_1, v_2, v_6, v_7\}$.
- *Partition Index Table (S):* This table facilitates subgraph connectivity queries, as depicted in Fig. 2(d), where the bipartite graph G consists of two subgraphs. To efficiently compute connected subgraphs, we employ a graph coloring technique to partition the bipartite graph. The vertex sets corresponding to each partition are then stored in the index table S.

To determine whether a graph is bipartite, we use a bipartite graph coloring method. The key idea is to divide the vertex set into two groups such that no two vertices within the same group are connected by an edge, whereas vertices in different groups are connected. This is achieved by: (1) Traversing each uncolored vertex in the graph. (2) Performing a depth-first search (DFS) or breadth-first search (BFS) on each uncolored vertex while assigning alternating colors (e.g., "0" and "1"). (3) If, during traversal, an edge connects two vertices of the same color, the graph is not bipartite. Otherwise, it is bipartite.

The above basic indexes primarily support (α, β)-core queries. To enable more flexible community searches, we extend these indexes to support (α, β)-WC and (α, β)-AWC queries.

Extended Index: Based on vertex attributes and edge weight values in the bipartite graph, we introduce three additional index tables:

- *Keyword Index Table (I_W):* This table stores lower-layer vertices associated with specific keywords. For

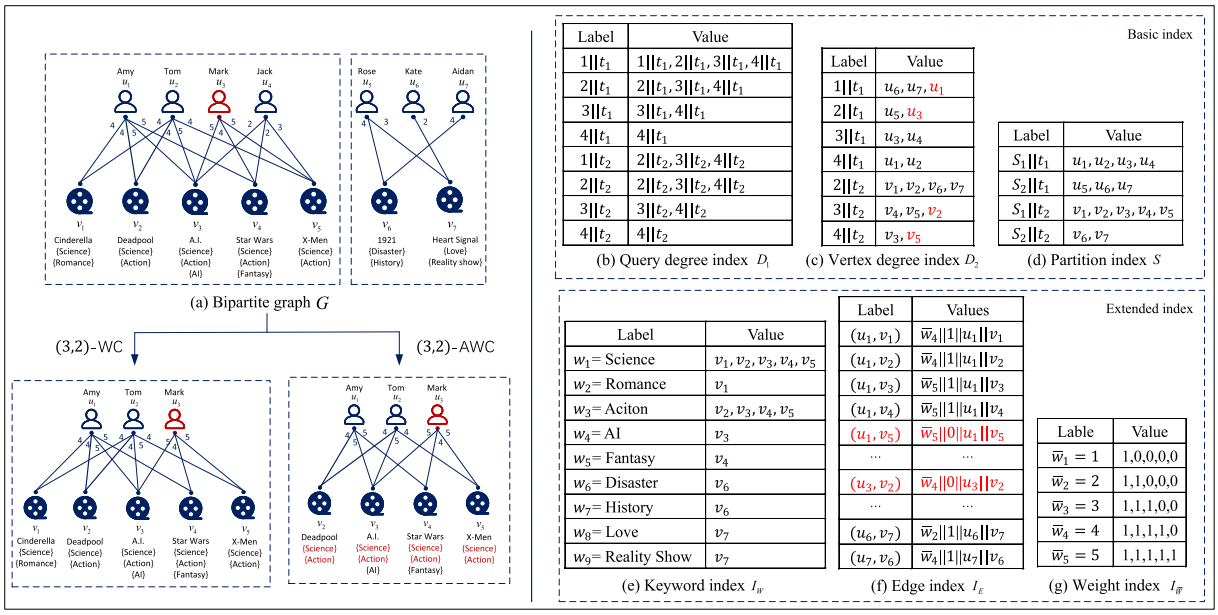


Fig. 2. An illustration of a bipartite graph and its corresponding index structures. (a) A bipartite graph supporting (α, β)-WC and (α, β)-AWC queries. (b)–(d) Basic index structures for (α, β)-core queries. (e)–(g) Extended index structures for (α, β)-WC and (α, β)-AWC queries.

example, as shown in Fig. 2(e), considering the keyword "w = Science", the relevant lower-layer vertices are $\{v_1, v_2, v_3, v_4, v_5\}$, which are stored in I_W.

- *Weight Index Table ($I_{\bar{W}}$):* This index assists in efficiently retrieving the minimum ciphertext. For instance, in Fig. 2(g), which represents the IMDB website, edge weights correspond to movie ratings ranging from $\{1, 2, 3, 4, 5\}$. When $\bar{w} = 1$, the corresponding 0-1 encoded array is computed using the 0-1 cipher encoding and stored in $I_{\bar{W}}(\bar{w}) = \{1, 0, 0, 0, 0\}$.
- *Edge Index Table (I_E):* This table stores metadata for each edge e, represented as a four-tuple: (weight, 1, upper-layer vertex, lower-layer vertex), where "1" serves as an auxiliary value for matrix construction in ciphertext environments. For example, considering the edge $e(u_1, v_1)$ in Fig. 2(f), the corresponding entry is $I_E(u_1||v_1) = \{4, 1, u_1, v_1\}$.

For (α, β)-core queries, the query degree index table D_1, the vertex degree index table D_2, and the partition index table S are primarily used. The retrieval process involves: (1) Fetching the D_2 index entries for degrees equal to α and β from D_1. (2) Extracting the set of query vertices $V = (U, L)$ that satisfy the degree constraints. (3) Filtering these vertices using the partition index table S. (4) Constructing an adjacency matrix based on the filtered vertex set.

For (α, β)-WC queries, the process extends (α, β)-core queries by further filtering edge weights using the weight index table $I_{\bar{W}}$, ensuring that the resulting subgraph satisfies both degree constraints and the maximum weight values.

Similarly, (α, β)-AWC queries further refine (α, β)-WC queries by incorporating keyword constraints. Using the keyword index table I_W, subgraphs are retrieved based on query keywords while maintaining both the degree and weight constraints.

Algorithm: The index construction process is executed by DO via the algorithm BuildIndex. This algorithm generates a comprehensive set of indexes $I = \{D_1, D_2, S, I_W, I_{\bar{W}}, I_E\}$, based on a bipartite graph G, a keyword set W, and a weight set $\bar{W}$. The complete algorithm is summarized in Algorithm 3.

First, the maximum degrees d_U and d_L for upper and lower layer vertices are computed (line 3). Then, the index tables D_1 and D_2 for upper-layer vertices are constructed (lines 4-6). The table D_1 stores degree values greater than or equal to i ($1 \le i \le d_U$). For instance, when $i = 1$, the values $\{1\|t_1, 2\|t_1, 3\|t_1, 4\|t_1\}$ are stored in $D_1(||t_1)$. Meanwhile, D_2 stores vertices with an exact degree of i. Similarly, D_1 and D_2 are constructed for lower-layer vertices (lines 7-9).

To partition G into connected subgraphs, a depth-first search (DFS) and bipartite graph coloring approach is applied. The partition index table S is then built based on the partition results (line 10), where σ denotes the number of connected subgraphs. For example, as illustrated in Fig. 2(a), the graph is divided into two partitions ($\sigma = 2$): $S'_1 = \{\{u_1, u_2, u_3, u_4\}, \{v_1, v_2, v_3, v_4, v_5\}\}$ and $S'_2 = \{\{u_5, u_6, u_7\}, \{v_6, v_7\}\}$. The index S is then structured based on these partitions, such that $S(1\|t_1) = S'_1(1)$ and $S(1\|t_2) = S'_1(2)$, where "1" denotes the partition number.

Next, the lower-layer vertices containing specific keywords $w \in W$ are recorded in the keyword index table I_W (line 11). Note that keywords are only considered for lower-layer vertices, but this can be adjusted as needed.

For weight encoding, the 0-1 encoding method is employed (lines 12-16). Consider the IMDB movie rating system with weight values in the range $\bar{W} = \{1, \ldots, 5\}$. For a specific rating $\bar{w} = 2$, a binary array $R_{\bar{w}}$ of length len = 5 is generated. If $i \le \bar{w}$ ($i \in \{1, \ldots, \text{len}\}$), a "1" is assigned to $R_{\bar{w}}(i)$; otherwise, a "0" is assigned. For instance, the encoding for $\bar{w} = 2$ results in $R_{\bar{w}} = \{1, 1, 0, 0, 0\}$.

Algorithm 3: BuildIndex$(G, W, \bar{W}) \rightarrow I$.

Input: Bipartite graph G, keyword set W and weight set $\bar{W}$.
Output: $I := \{D_1, D_2, S, I_W, I_{\bar{W}}, I_E\}$.
1 Parse G as (U, L, E);
2 Initialize dictionaries D_1, D_2, S, I_W, $I_{\bar{W}}$ and I_E;
3 $d_U \leftarrow \mathsf{deg}_{\mathsf{max}}(U)$; $d_L \leftarrow \mathsf{deg}_{\mathsf{max}}(L)$;
4 **for** $i = 1 : d_U$ **do**
5 For $j \in (i, \cdots, d_U)$, store $(j||t_1)$ to $D_1(i||t_1)$;
6 For $u \in U$, if $\mathsf{deg}(u) = i$, then $D_2(i||t_1) \leftarrow u$;
7 **for** $i = 1 : d_L$ **do**
8 For $j \in (i, \cdots, d_L)$, store $(j||t_2)$ to $D_1(i||t_2)$;
9 For $v \in L$, if $\mathsf{deg}(v) = i$, then $D_2(i||t_2) \leftarrow v$;
10 Using coloring method to judge and split multiple bipartite graphs, and store $S(i||t_1) \leftarrow u \in U_{i\in\sigma}$ and $S(i||t_2) \leftarrow v \in L_{i\in\sigma}$.
11 For each keyword $w \in W$, compute and store the node $v \in L$ with that keyword into index I_W.
12 **for** $\bar{w} \in \bar{W}$ **do**
13 $R_{\bar{w}} \leftarrow$ an empty array;
14 For $i \in |\bar{W}|$, if $i \leq \bar{w}$, store '1' to $R_{\bar{w}}(i)$, otherwise store '0' to $R_{\bar{w}}(i)$;
15 $I_{\bar{W}}(\bar{w}) \leftarrow R_{\bar{w}}$;
16 Get edges $e \in E$ with $\bar{w}$, store $(\bar{w}, 1, u, v)$ to $I_E(u||v)$;
17 Add some obfuscating edges into I_E and then update D_2 accordingly.
18 Output $I := \{D_1, D_2, S, I_W, I_{\bar{W}}, I_E\}$.

Algorithm 4: GenKey$(1^\lambda) \rightarrow K$.

Input: A security parameter λ.
Output: $K = \{k_1, k_2, k_3, k_4, k_5\}$.
1 **for** $i = 1 : 3$ **do**
2 $k_i \leftarrow f(s_i, \lambda)$;
3 $k_4 \leftarrow \Phi.\mathsf{Gen}(1^\lambda)$;
4 $k_5 \leftarrow \Pi.\mathsf{Gen}(1^\lambda)$;
5 Output $K = \{k_1, k_2, k_3, k_4, k_5\}$.

Finally, a set of obfuscating edges is incorporated into I_E, and index D_2 is updated accordingly (line 17). As illustrated in Fig. 2(c) and (f), u_1 and v_5 are not originally connected, but we add such an edge to obfuscate the edge table. Specifically, a virtual edge $(\bar{w}, 0, u_1, v_5)$ is inserted to prevent an adversary from deducing vertex degrees.

B. (α, β)-Core Query Method

Core Method: First, we retrieve an initial set of vertices that satisfy the query constraints utilizing the index tables and construct an adjacency matrix. Then, by performing "row" and "column" addition operations on the adjacency matrix, we efficiently and accurately compute the (α, β)-core. The detailed steps for computing the (α, β)-core based on the adjacency matrix are as follows.

a) *Perform "row" operation:* Compute the sum of elements in each row. For example, consider the upper-layer vertices $U = \{u_1, u_2, u_3, u_4\}$ in Fig. 3(g). The result of the "row" operation is $\alpha' = \{4, 4, 3, 3\}$.

b) *Compare with α:* Compare each α'_i obtained from the "row" operation with α, filtering out upper-layer vertices that do not meet the α condition. For instance, if $\alpha = 3$, then comparing $\alpha' = \{4, 4, 3, 3\}$ with α, we retain the vertices where $\alpha'_i \geq \alpha$ and discard the rest.

c) *Perform "column" operation on filtered vertices:* Compute the sum of elements in each column for the remaining vertices. For example, considering the lower-layer vertices $L = \{v_1, v_2, \ldots, v_5\}$ in Fig. 3(g), the "column" operation yields $\beta' = \{2, 2, 4, 3, 3\}$.

d) *Compare with β:* Compare each β'_i with β, filtering out lower-layer vertices that do not meet the β condition. If $\beta = 2$, then comparing $\beta' = \{2, 2, 4, 3, 3\}$ with β, we retain vertices where $\beta'_i \geq \beta$ and discard the rest.

e) *Iterate for accuracy:* To ensure accurate (α, β)-core query results, repeat steps (a)-(d) for the refined set of vertices. It terminates once both "row" and "column" operations simultaneously satisfy the query conditions.

Extended Method: Based on the core method, we introduce two enhanced queries: (α, β)-WC and (α, β)-AWC.

For (α, β)-WC, additional filtering is applied to edge weights based on the results of the (α, β)-core query. This process identifies a subgraph that meets the degree constraints while maximizing edge weights. As illustrated in Fig. 3(g), the minimum edge weight in the (α, β)-core subgraph is determined as $\bar{w} = 2$. Subsequently, edges with weight $\bar{w} = 2$ are removed by modifying the corresponding matrix values. For example, if the weight of edge $e = (u_4, v_3)$ is 2, its matrix value is updated to 0. The "row" and "column" operations are then re-applied following the same steps as the (α, β)-core query.

For (α, β)-AWC, an additional attribute-based filtering step is applied when selecting the vertex set. The refined vertex set is used to construct an adjacency matrix, and the (α, β)-WC operations are performed. The specific process is illustrated in Fig. 3(h).

VI. Our STE-BG Construction

The STE-BG scheme consists of five components, where GenKey, EncData, and Token are executed by the DO.

A. GenKey$(1^\lambda) \rightarrow K$

Initially, the DO inputs the security parameter λ and runs the GenKey algorithm to generate a set of keys $K = \{k_1, k_2, k_3, k_4, k_5\}$. Here, k_4 serves as the key for the SKE scheme, while k_5 is used for the SAHE scheme, enabling complex operations on encrypted data. The detailed key generation process is outlined in Algorithm 4.

B. EncData$(K, G, \mathbf{M}) \rightarrow (\gamma, \mathbf{c})$

The DO inputs a set of system keys K, a bipartite graph G, and data items $\mathbf{M}$. It then executes the EncData algorithm

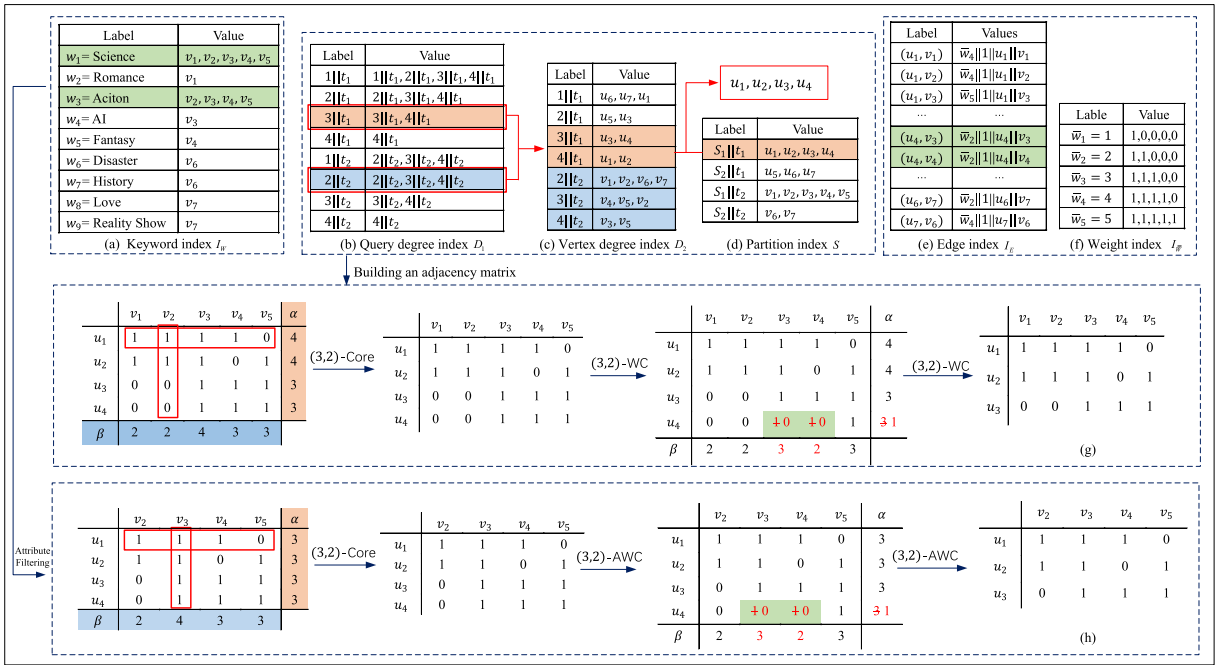


Fig. 3. Different query operations on the index lists. (a)-(f) Multiple indexed tables for complex queries. (g)-(h) Execution of the (3,2)-WC and (3,2)-AWC arithmetic processes.

to generate a secure index γ and ciphertext $\mathbf{c}$, as outlined in Algorithm 5.

First, the BuildIndex algorithm constructs a set of plaintext indices (line 3). Then, it employs pseudorandom functions and the SKE scheme to encrypt the indices $\{D_1, D_2, S, I_W, I_{\bar{W}}, I_E\}$, producing the secure indices $\{T_{D_1}, T_{D_2}, T_\theta, T_W, T_E\}$ (lines 4-13).

For instance, to construct T_{D_1}, a pseudorandom function encrypts the keys in D_1 as $H_{k_1}(d)$, while symmetric encryption secures the values, yielding $(\Phi.\mathsf{Enc}(k_4, H_{k_1}(i)_{i\in D_1(d)})) \oplus F_{k_3}(d)$. The difference between the encrypted value and $F_{k_3}(d)$ ensures uniform ciphertext length for enhanced security.

The detailed encryption process is illustrated in Fig. 4.

C. *Token* $(K, \alpha, \beta, qu, qw, t) \to \tau$

The DO generates a set of search tokens based on the users query request. Here, α and β represent vertex degree constraints, qu denotes the query vertices, $\mathrm{q_w}$ represents the query keywords, and t indicates the query type. Specifically, $\mathrm{t} = 1$ corresponds to executing an (α, β)-AWC query, while $\mathrm{t} = 2$ corresponds to an (α, β)-WC query. The key difference between these two queries lies in whether keyword constraints are applied. The detailed process is described in Algorithm 6

D. *Search* $(\gamma, \tau) \to (A^*, J)$

The Search algorithm performs privacy-preserving community search based on the (α, β)-core. First, it executes "row" and "column" summation operations on the adjacency matrix. The results are then compared with the vertex degree constraints $(\alpha$ and $\beta)$ to identify subgraphs that contain the query vertices while satisfying the structural cohesion conditions. Subsequently, weight trimming is applied to facilitate (α, β)-WC and (α, β)-AWC queries. The detailed steps are outlined in Algorithm 7.

Algorithm 5: $\mathsf{EncData}(K, G, \mathbf{M}) \to (\gamma, \mathbf{c})$.

Input: $K, G, \mathbf{M}$.
Output: A set of secure indexes γ and ciphertext $\mathbf{c}$.
1 Parse G as (U, L, E);
2 Parse $\mathbf{M}$ as $(\mathbf{m}, W, \bar{W})$;
3 $I \leftarrow \mathsf{BuildIndex}(G, W, \bar{W})$;
4 $\mathbf{m}^* = [\,]$ and $\mathbf{m}^* \leftarrow (m_j)_{j=\pi(i)}$;
5 For $1 \le \zeta \le n$, let $c_\zeta \leftarrow \Phi.\mathsf{Enc}(k_4, m^*_\zeta)$, where $(m^*_\zeta \in \mathbf{m}^*)_{\zeta\in[1,|\mathbf{m}^*|]}$;
// Encrypting indexed tables.
6 For each $d \in D_1$, store $(\Phi.\mathsf{Enc}(k_4, H_{k_1}(i)_{i\in D_1(d)})) \oplus F_{k_3}(d)$ in T_{D_1} with search key $H_{k_1}(d)$;
7 For each $d' \in D_2$, store $H_{k_2}(i)_{i\in D_2(d')}$ in T_{D_2} with search key $H_{k_1}(d')$;
8 For each $s \in S$, store $H_{k_2}(i)_{i\in S(s)}$ in T_S with search key $H_{k_1}(s)$;
9 **for** $e^* \in I_E$ **do**
10 $(u, v, \bar{w}) \leftarrow I_E(e^*)$;
11 $T_E(H_{k_2}(u) \oplus H_{k_2}(v)) \leftarrow \theta_{\bar{w}} \| \Pi.\mathsf{Enc}(k_5, 1) \| \pi(u) \| \pi(v) \oplus H_{k_2}(u)$, where $\theta_{\bar{w}}$ is the y-dimensional 0-1 encoded array.
12 For each $\xi \in I_W$, store $H_{k_2}(i)_{i\in I_W(\xi)}$ in T_W with search key $H_{k_1}(\xi)$;
13 Output $\gamma := \{T_{D_1}, T_{D_2}, T_S, T_\theta, T_W, T_E\}$ and $\mathbf{c} = (c_1, \cdots, c_n)$.

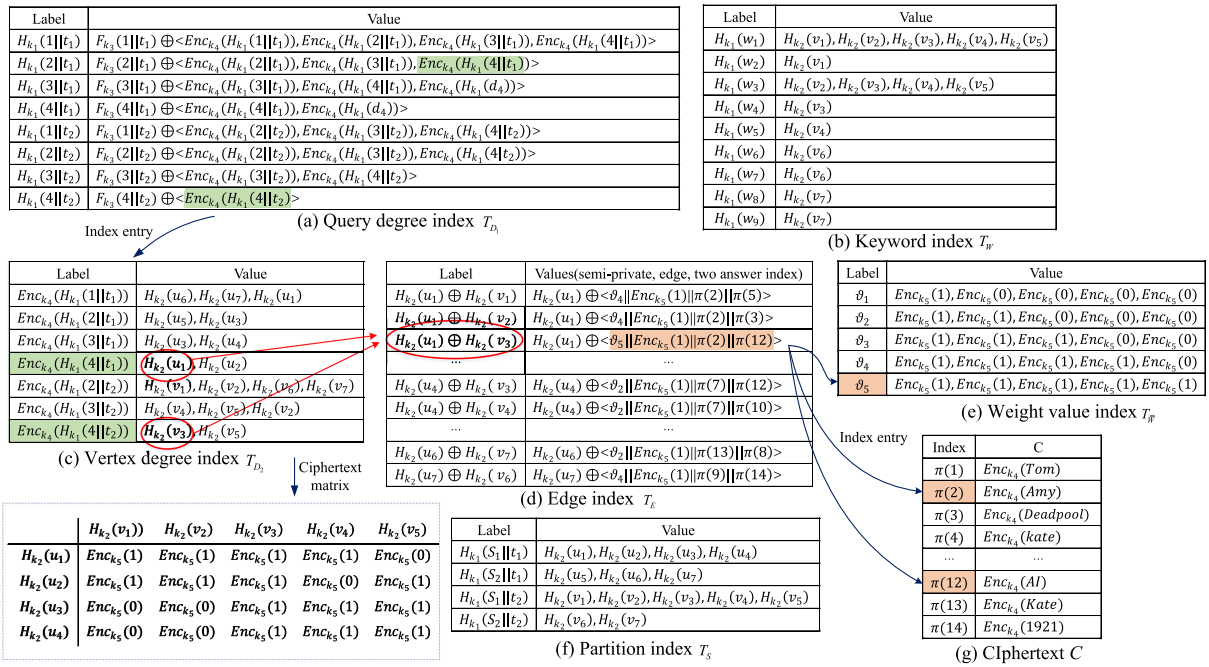


Fig. 4. Privacy-preserving query scheme based on encrypted index table implementation.

Algorithm 6: $\mathsf{Token}(K, \alpha, \beta, q_u, q_w, t) \rightarrow \tau$.

Input: $K, \alpha, \beta, q_u, q_w, t$.
Output: τ.

1 Parse K to $(k_1, k_2, k_3, k_4, k_5)$, parse q_u to u;
2 Compute and store $\{H_{k_1}(\alpha||t_1), H_{k_1}(\beta||t_2), F_{k_3}(\alpha||t_1), F_{k_3}(\beta||t_2), H_{k_2}(u), \Pi.\mathsf{Enc}(k_5, \alpha), \Pi.\mathsf{Enc}(k_5, \beta)\}$ to τ_1.
3 **if** $t == 1$ **then**
4 $\quad \tau_2 \leftarrow H_{k_1}(q_w)$;
5 $\quad \tau \leftarrow \{t, \tau_1, \tau_2\}$;
6 **if** $t == 2$ **then**
7 $\quad \tau \leftarrow \{t, \tau_1\}$;
8 Output τ.

Lines 1-3 describe how the cloud server (CS) retrieves data item sets R_1 and R_2 from the secure index T_{D_1} using the search token τ, where the search keys are $H_{k_1}(\alpha||t_1)$ and $H_{k_1}(\beta||t_2)$. Line 4 utilizes elements from R_1 and R_2 as index entries for T_{D_2} (refer to Fig. 4(a) and (c)), obtaining the upper-layer vertex set B_1 and lower-layer vertex set B_2 that meet the degree constraints. Line 5 ensures that when the search type $t = 1$, elements in B_2 that do not match $T_w(\tau_2)$ are filtered out. Line 6 constructs the adjacency matrix A and a set of weight values $\bar{W}$ based on B_1, B_2, and T_E (see the ciphertext matrix in Fig. 4(c)).

Lines 8-12 compute the (α, β)-core that satisfies the constraints. Specifically, the algorithm performs "row" and "column" addition operations on matrix A and securely compares the results with the vertex constraints. If a computed sum is below the constraints, the corresponding index a_{ij} is stored in B_2'. Simultaneously, vertices in B_1 that fail to meet the degree constraints are filtered out. This process iterates until a subgraph is obtained that satisfies the constraints and includes the query vertices. If the query vertices do not exist in B_1 and B_2, the iteration halts, returning a message indicating that no matching subgraph was found.

Line 13 removes vertices that do not satisfy the constraints and reconstructs a new adjacency matrix A'. Lines 14-24 focus on weight trimming to refine the subgraph further. First, the minimum weight value is determined using a secure minimum-value protocol. Next, ciphertext values corresponding to edges with this minimum weight in A' are modified. The algorithm then performs "row" and "column" addition operations on the updated matrix, following the same steps as the (α, β)-core computation. Through iterative refinement, the final matrix A^* is obtained.

Ultimately, the algorithm outputs A^* along with a set of ciphertext indices as the final search results.

E. $\mathsf{DecData}(K, A^*, J, \mathbf{c}) \rightarrow (\hat{A}, m_{\pi^{-1}(J_1)}, m'_{\pi^{-1}(J_2)})$

The user utilizes J as a pointer to retrieve the corresponding ciphertext value $\mathbf{c}[J]$. Then, the user employs the symmetric keys k_4 and k_5, authorized by DO, to decrypt the ciphertext and the encrypted matrix, respectively. This process yields the plaintext information of upper-layer vertices $m_{\pi^{-1}(J_1)}$, lower-layer vertices $m'_{\pi^{-1}(J_2)}$, and the adjacency matrix $\hat{A}$ of the subgraph. For example, using $J = (4, 5)$ as a pointer, $\mathbf{c} = (c_1, \ldots, c_n)$ are retrieved and decrypted as $\mathsf{DecData}(k_5, c_4) \rightarrow u_3$, $\mathsf{DecData}(k_5, c_5) \rightarrow v_1$. The specific details are outlined in Algorithm 8.

Algorithm 7: $\mathsf{Search}(\gamma, \tau) \rightarrow (A^*, J)$.

Input: A query token τ and a set of indexes γ.
Output: A^*, J.
1 Parse τ as (t, τ_1, τ_2) and parse γ as $(T_{D_1}, T_{D_2}, T_S, T_\theta, T_E, T_W)$;
2 Parse τ_1 as $H_{k_1}(\alpha||t_1), H_{k_1}(\beta||t_2), F_{k_3}(\alpha||t_1), F_{k_3}(\beta||t_2), H_{k_2}(u), \Pi.\mathsf{Enc}(k_5, \alpha)$ and $\Pi.\mathsf{Enc}(k_5, \beta)$;
3 Store $T_{D_1}(H_{k_1}(\alpha||t_1)) \oplus F_{k_3}(\alpha||t_1)$ and $T_{D_2}(H_{k_1}(\beta||t_2)) \oplus F_{k_3}(\beta||t_2)$ to R_1 and R_2;
4 For each $R_{1i} \in R_1$ and $R_{2i} \in R_2$, store $T_{D_2}(R_{1i})$ and $T_{D_2}(R_{2i})$ to B_1 and B_2 ;
5 If $t == 1$, remove elements from B_2 that do not exist in $T_w(\tau_3)$;
6 Build the adjacency matrix A and weights $\bar{W}$ based on B_1, B_2, and T_E;
7 Initial two temporal arrays $B_1' = B_1$, $B_2' = B_2$;
// Performing determinant operations on matrices.
8 **while** $H_{k_2}(u) \in B_1$ or $H_{k_2}(u) \in B_2$ **do**
9 **for** $a_i \in A$ **do**
10 $\mathsf{sum} = a_{i1}$ and for $j \in |a_i|$ and $j \notin B_2'$, compute $\mathsf{sum} = \mathsf{sum} + a_{ij}$;
11 Compute $\mathsf{SCP}(\mathsf{sum}, \Pi.\mathsf{Enc}(k_5, \beta))$ and if $\mathsf{sum} < \Pi.\mathsf{Enc}(k_5, \beta)$, store i in B_2' and remove $B_1(i)$.
12 Repeat lines 10-12, replace $\{B_1, a_{i1}, B_2, B_2', a_{ij}\}$ on lines 10-11 with $\{B_2, a_{1i}, B_1, B_1', a_{ji}\}$, and replace $(\Pi.\mathsf{Enc}(k_5, \beta), B_2', B_1(i)$ with $(\Pi.\mathsf{Enc}(k_5, \alpha)), B_1', B_2(i))$.
13 Based on B_1' and B_2', filter and construct the latest matrix A' for A that satisfies α and β conditions, and filter and store B_1', B_2' to new arrays B_1^*, B_2^*.
14 $\hat{B}_1' = B_1^*$; $\hat{B}_2' = B_2^*$; $\mathsf{w_{min}} \leftarrow \mathsf{SMCP}(\bar{W})$;
15 Inital two temporal arrays upd_1 and upd_2;
// Obtain significant subgraphs.
16 **while** $H_{k_2}(u) \in B_1^*$ or $H_{k_2}(u) \in B_2^*$ **do**
17 **for** $b \in B_1^*$, $b' \in B_2^*$ **do**
18 **if** $R_1' = \mathsf{w_{min}}$ **then**
19 Update the value of $a_{bb'}^* \in A'$ to R_2';
20 $\mathsf{upd}_1 \leftarrow b$, $\mathsf{upd}_2 \leftarrow b'$;
21 Repeat lines 10-12, replace B_2, B_1 with upd_2, upd_1;
22 Remove $\mathsf{w_{min}}$ in $\bar{W}$ and get new minimal weigh $\mathsf{w_{min}} \leftarrow \mathsf{SMCP}(\bar{W})$;
23 Based on upd_1, upd_2, filter and construct the latest matrix A^* for A' that satisfies significant (α, β)-WC.
24 For each $b \in \mathsf{upd}_1$, $b' \in \mathsf{upd}_2$, compute $R^* \leftarrow b \oplus T_E(b||b')$ and store $R^*[3]$ and $R^*[4]$ to J_1 and J_2;
25 Output $A^*, J := \{J_1, J_2\}$.

Algorithm 8: $\mathsf{DecData}\ (K^*, A^*, J, \mathbf{c}) \rightarrow (\hat{A},\ m_{\pi^{-1}(J_1)}, m'_{\pi^{-1}(J_2)})$.

Input: $K^*, A^*, J, \mathbf{c}$.
Output: $\hat{A}, m_{\pi^{-1}(J_1)}, m'_{\pi^{-1}(J_2)}$.
1 Parse K^* as k_4, k_5 and J as J_1, J_2;
2 Initial a matrix $\hat{A}$;
// Decrypt to get vertex information.
3 Compute $m_{\pi^{-1}(J_1)} = \Phi.\mathsf{Dec}(k_4, \mathbf{c}[J_1])$;
4 Compute $m'_{\pi^{-1}(J_2)} = \Phi.\mathsf{Dec}(k_4, \mathbf{c}[J_2])$;
5 For each $a_{ij} \in A$, compute and store $\Pi.\mathsf{Dec}(k_5, a_{ij})$ to $\hat{a}_{ij} \in \hat{A}$;
6 Output $\hat{A}, m_{\pi^{-1}(J_1)}, m'_{\pi^{-1}(J_2)}$.

VII. Security Analysis

The key difference between the proposed scheme [10] and our approach lies in their defense capabilities. The scheme in [10] only mitigates passive attacks and fails to defend against CQA2, making it susceptible to information leakage under active attacks. In contrast, our STE-BG scheme is fundamentally based on the structured encryption framework proposed in [16], and its security definition is fully compatible with the CQA2-security model. By leveraging structured encryption techniques, our scheme seamlessly integrates the CQA2-security model introduced in [16].

To ensure the robustness of our scheme, we rely on two additional properties of the underlying simple structured encryption scheme. First, the $\mathcal{L}_1$ leakage associated with the encrypted data structure and ciphertexts must depend exclusively on the data items, rather than on any semi-private data. Additionally, query tokens may exhibit varying leakage patterns when interacting with complex data structures. This property, referred to as *Chainability* in [16], is inherently satisfied by our STE-BG scheme.

Definition 9 (Chainability): A structured encryption scheme $\Sigma = (\mathsf{GenKey}, \mathsf{EncData}, \mathsf{Token}, \mathsf{Search}, \mathsf{DecData})$ with $(\mathcal{L}_1, \mathcal{L}_2)$ leakage is said to be chainable if it satisfies CQA2-security and the leakage functions $\mathcal{L}_1$ and $\mathcal{L}_2$ adhere to the following properties:

- *Semi-private independence:* There exists a function $\mathcal{L}_1'$ such that
$$\mathcal{L}_1(\gamma, (m, \vartheta)) = \mathcal{L}_1'(\gamma, m).$$
- *Order independence:* There exists a transformation $\mathcal{T}$ such that for any data structure γ, any set of queries $q_1, \ldots, q_l$, and any permutation p,
$$\mathcal{T}(\mathcal{L}_2(\gamma, q_1), \ldots, \mathcal{L}_2(\gamma, q_l), p) = (\mathcal{L}_2(\gamma, q_{p(1)}), \ldots, \mathcal{L}_2(\gamma, q_{p(l)})).$$

Theorem 1: Assuming that the employed PRFs and PRPs are secure, and that the underlying primitives SKE and SAHE are CPA-secure, the proposed scheme STE-BG achieves $(\mathcal{L}_1, \mathcal{L}_2)$-security against adaptive chosen-query attacks (CQA2). Consequently, STE-BG is a chainable structured encryption scheme according to Definition 9.

$$\mathcal{L}_1(G, \mathbf{M}) = (n, |\gamma|, |\mathbf{c}|).$$
$$\mathcal{L}_2(\gamma, q) = (\mathrm{QP}(q), \mathrm{AP}(q)).$$

Here, q denotes a query of the form $q = (\alpha, \beta, q_u, q_w, t)$.

Proof: We prove the theorem via a sequence of hybrid games between the real experiment $\mathbf{Real}_{\mathcal{A}}(\lambda)$ and the ideal experiment $\mathbf{Ideal}_{\mathcal{A},\mathcal{S}}(\lambda)$, where $\mathcal{A}$ is a probabilistic polynomial-time (PPT) adversary, $\mathcal{S}$ is a simulator, and $\mathcal{B}$ is the challenger.

Game_0: This is the real experiment $\mathbf{Real}_{\mathcal{A}}(\lambda)$. The challenger $\mathcal{B}$ samples keys $K \leftarrow \mathsf{GenKey}(\lambda)$, where $K =$

$\{k_1, \ldots, k_5\}$. The adversary $\mathcal{A}$ submits a dataset $(G, \mathbf{M})$ and receives $(\gamma, \mathbf{c}) \leftarrow \mathsf{EncData}(K, G, \mathbf{M})$. It then adaptively issues queries $q = (\alpha, \beta, q_u, q_w, t)$ and obtains tokens $\tau \leftarrow \mathsf{Token}(K, \alpha, \beta, q_u, q_w, t)$. Finally, $\mathcal{A}$ outputs a bit b.

Game_1*:* The challenger generates K as in Game_0, but replaces the real encrypted structures with simulated ones. Specifically, upon receiving $(G, \mathbf{M})$, the adversary obtains $\mathcal{S}(\mathcal{L}_1(G, \mathbf{M}))$. For each query q, let $R(q)$ denote the accessed index set corresponding to the index list T_ϑ and define $y_q := (\vartheta_i)_{i \in R(q)}$. The token is generated as

$$\tau \leftarrow \mathcal{S}(\mathcal{L}_2(\gamma, q), y_q).$$

Transition from Game_0 *to* Game_1*:* If there exists a PPT adversary $\mathcal{A}$ that distinguishes these two games with non-negligible advantage, then one can construct a reduction that breaks either the pseudorandomness of the PRFs/PRPs or the CPA security of SKE and SAHE. Hence, under the assumed security of these primitives, the two games are computationally indistinguishable.

Game_2*:* This game is identical to Game_1, except that tokens are generated in a stateless manner. For each query q, define:

- For all $i \in R(q)$, $\vartheta_i = H_{k_1}(q)$;
- $\tau \leftarrow \mathcal{S}(\mathcal{L}_2(\gamma, q), y_q)$, where $y_q := (\vartheta_i)_{i \in R(q)}$.

Since token generation does not introduce additional leakage and remains consistent with $\mathcal{L}_2$, Game_2 is indistinguishable from Game_1.

Game_3*:* In this game, both the encrypted data structures $(\gamma, \mathbf{c})$ and all query tokens are entirely generated by the simulator using only $\mathcal{L}_1(G, \mathbf{M})$ and $\mathcal{L}_2(\gamma, q)$. This corresponds exactly to the ideal experiment $\mathbf{Ideal}_{\mathcal{A},\mathcal{S}}(\lambda)$.

Transition from Game_2 *to* Game_3*:* By the CPA security of SKE and SAHE, and the indistinguishability of PRF/PRP outputs from random, the simulated ciphertexts and tokens are computationally indistinguishable from the real ones. Therefore, Game_2 and Game_3 are indistinguishable.

Combining the above hybrids, we have

$$\mathsf{Game}_0 \approx \mathsf{Game}_1 \approx \mathsf{Game}_2 \approx \mathsf{Game}_3,$$

which implies that $\mathbf{Real}_{\mathcal{A}}(\lambda)$ and $\mathbf{Ideal}_{\mathcal{A},\mathcal{S}}(\lambda)$ are indistinguishable. Hence, STE-BG satisfies $(\mathcal{L}_1, \mathcal{L}_2)$-security under adaptive-query attacks (CQA2). □

VIII. Quantitative Analysis of Leakage

The query process in the proposed scheme generates three types of observable information: (1) the set of neighbors of the queried node, (2) the weight information associated with the edges, and (3) the number of query results.

To enhance privacy, the system introduces virtual edges and synthetic weights. As a result, the adversary cannot distinguish between real and virtual edges from the query output, leading to increased uncertainty in reconstructing the original graph structure.

To quantify this uncertainty and evaluate the scheme's privacy strength, we perform a combinatorial analysis of the reconstruction space size based on neighbor structure, edge weights, and the virtual edge strategy under the structured encryption setting. This yields the following entropy estimation model.

Let d_v denote the true degree of node v, and d'_v its observed degree (including virtual edges). Let S_w represent the total sum of all edge weights, and let $e' = \sum_v d'_v/2$ denote the total number of (real + virtual) edges. The size of the reconstruction space can be approximated as:

$$|RS(G)| \approx \prod_{v=1}^{n} \binom{d'_v}{d_v} \cdot \binom{S_w - 1}{e' - 1}. \tag{8}$$

Accordingly, the leakage inversion entropy is estimated by:

$$\text{LeakInvH}(G, \Lambda) = \log \left(\prod_{v=1}^{n} \binom{d'_v}{d_v} \cdot \binom{S_w - 1}{e' - 1} \right). \tag{9}$$

Here, the binomial term $\binom{d'_v}{d_v}$ captures the adversary's uncertainty in selecting the correct set of real neighbors among all observed neighbors (i.e., choosing d_v unordered, distinct elements from d'_v). The term $\binom{S_w - 1}{e' - 1}$ quantifies the number of possible ways to distribute the total weight S_w across all e' edges.

Our design intentionally injects virtual edges and randomized weights into query responses. Since the ciphertext and result formats do not distinguish fake edges from real ones, this effectively enlarges the adversary's candidate space for reconstruction. As more virtual edges are introduced, the observed degree d'_v increases while the true degree d_v remains unchanged, thereby expanding the combinatorial term $\binom{d'_v}{d_v}$ and increasing the estimated entropy LeakInvH.

In conclusion, this setting has been rigorously analyzed via a game-based simulation and leakage quantification under the CQA2 model, demonstrating that the proposed scheme STE-BG achieves strong privacy guarantees.

IX. Performance Evaluation

Experimental Environment: The proposed graph encryption scheme was implemented in Java, utilizing the IDEA development tool. Experiments were conducted on a system equipped with an AMD Ryzen7 5800H 3.2 GHz processor, 16 GB RAM, and a 4 GB NVIDIA GeForce GTX 1650 GPU, running Windows 11 (64-bit). To ensure the reliability of results, each experiment was averaged over 10 trials.

Furthermore, we employed the Symmetria2 prototype system developed by Savvides et al. [31], which utilizes the SAHE scheme for data encryption. The PRF was instantiated using HMAC-SHA-256 with a security parameter $\lambda = 256$.

Datasets: We evaluated our scheme using three real-world datasets: DoubanMovies-small (DB),[1] MovieLens-1 M (ML),[2] and Hetrec (HT).[3] Each dataset represents a user-movie (or anime) rating network, with rating levels ranging from 1 to 5. Table II provides detailed dataset statistics, where $|U|$ and $|L|$ denote the number of vertices in layers U and L, respectively; $|E|$ represents the number of edges; $\alpha_{\max}$ and $\beta_{\max}$ correspond to the maximum degrees of vertices in layers U and L, respectively; and KT indicates the number of keyword types.

[1] https://moviedata.csuldw.com/.
[2] https://grouplens.org/datasets/movielens/1 m/.
[3] https://grouplens.org/datasets/hetrec-2011/.

TABLE II
DATASET STATISTICS

Name	$\|U\|$	$\|L\|$	$\|E\|$	$\alpha_{\max}$	$\beta_{\max}$	KT
DB	1142	1066	4086	147	101	12
ML	6040	2758	756975	1688	3428	19
HT	2113	10109	855598	3410	1670	21

TABLE III
EXPRESSIVENESS COMPARISON

Scheme	NComb.	(α, β)-Core	(α, β)-WC	(α, β)-AWC	Pri.
[24]	×	✓	×	×	×
[8]	×	✓	✓	×	×
[10]	×	✓	×	×	✓
Ours	✓	✓	✓	✓	✓

Comparison Algorithms: We compare our scheme against the following state-of-the-art approaches:

- BiCore-Index [24]: Constructs a compact BiCore-Index to efficiently process (α, β)-core queries.
- SCS-Index [8]: Introduces the significant (α, β)-WC model, leveraging two novel indices, I_δ and I_v, along with the SCS-Peel algorithm for efficient retrieval.
- PBG-Index [10]: Employs privacy-preserving techniques for secure and efficient bipartite (α, β)-core queries.
- STE-BG *(Our Scheme):* Incorporates structured encryption with index-based optimizations, supporting (α, β)-core, (α, β)-WC, and (α, β)-AWC queries.

Since the token overhead is negligible ($O(1)$ complexity), and DecData overhead primarily depends on the query result size, we evaluate the schemes based on four key aspects: expressiveness, storage overhead, index construction time, and search efficiency.

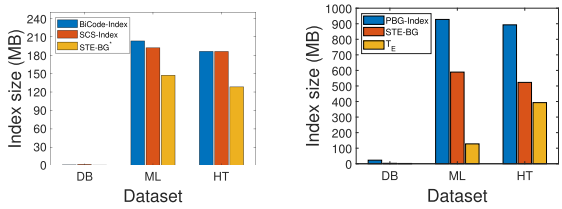
A. Expressiveness

Table III provides a comparative analysis of the expressive power of different schemes. Unlike BiCore-Index, SCS-Index, and PBG-Index, which require precomputing and storing multiple (α, β) combinations in index structures, our scheme dynamically constructs a temporary adjacency matrix based on query conditions, significantly reducing preprocessing and index maintenance overhead. Here "NComb." denotes there is no (α, β) combinations and "Pri." means privacy-preserving.

B. Storage Overhead

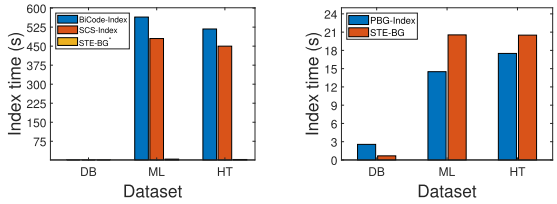
Storage overhead primarily depends on the size of the indexes. Initially, we construct the plaintext indexes $I = \{D_1, D_2, I_E, I_W, I_{\bar{W}}\}$ for the bipartite graph G using Algorithm BuildIndex. Subsequently, the secure indexes $\gamma = \{T_{D_1}, T_{D_2}, T_E, T_\vartheta, T_w\}$ are generated using Algorithm EncData. To assess the storage overhead, we analyze the sizes of both the plaintext and secure indexes. In this context, STE-BG* represents the construction of index I in a plaintext setting.

Plaintext Index Size: As shown in Fig. 5(a), the index size of STE-BG* is significantly smaller than that of other schemes. This reduction in storage overhead is due to our approach, which avoids storing all (α, β) combinations. Instead, our scheme efficiently supports community search by establishing relationships between vertex degrees, nodes, and edge tables.



(a) Plaintext index size (b) Secure index size

Fig. 5. Comparison of index sizes across different datasets.



(a) Plaintext index time (b) Secure index time

Fig. 6. Comparison of index construction time across different datasets.

In the ML dataset, the index sizes for BiCode-Index, SCS-Index, and STE-BG* are 203.12 MB, 192 MB, and 147.2 MB, respectively. Additionally, since index construction in all three schemes depends on vertex degrees, the index size grows proportionally as vertex degrees increase. These results highlight the advantage of STE-BG* in reducing storage overhead.

Secure Index Size: Since BiCode-Index and SCS-Index prioritize efficient indexing while disregarding data security, we compare the secure index sizes only with PBG-Index.

As illustrated in Fig. 5(b), the secure index size of PBG-Index is larger than that of STE-BG. This is because PBG-Index stores two sets for each α and constructs separate edge tables for each (α, β) combination. Each row in an edge table consists of six tuples, and the scheme constructs edge tables for $\alpha_{\max}$ columns, leading to a significant increase in index size as $\alpha_{\max}$ grows.

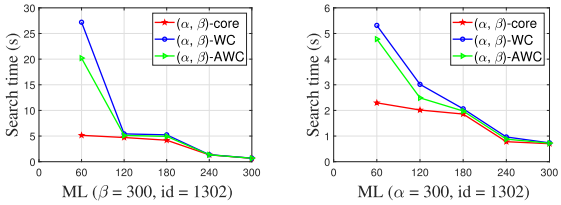
In contrast, STE-BG encrypts only the relationships between vertices and degrees along with the edge table, without storing each (α, β) combination separately. Consequently, in the ML dataset, the secure index sizes of PBG-Index and STE-BG are 927.7 MB and 589.2 MB, respectively.

Moreover, the secure index size of STE-BG is primarily determined by the size of the edge table T_E. As the graph size increases, so does the secure index size. In the HT dataset, the secure index sizes of STE-BG and T_E are 523 MB and 392.9 MB, respectively.

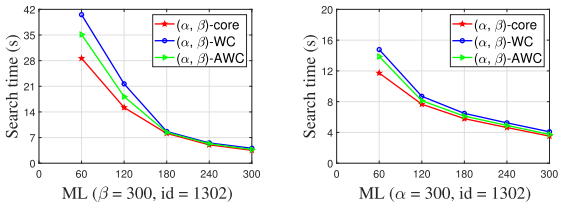
C. Index Construction Time

Plaintext Index Construction Time: As illustrated in Fig. 6(a), STE-BG* demonstrates a significant advantage over other schemes in terms of index construction time.

This efficiency stems from the fact that BiCode-Index and SCS-Index require a depth-first traversal of the graph to identify all possible (α, β) combinations. In contrast, STE-BG* constructs indices by leveraging vertex degrees and the maximum

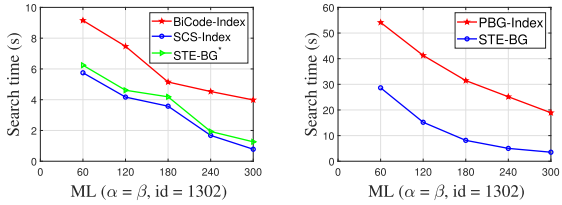


(a) Three query types with varying α (b) Three query types with varying β



(c) Ciphertext queries with varying α (d) Ciphertext queries with varying β

Fig. 7. Search time under different α and β values in plaintext and ciphertext environments.



(a) (α, β)-core in plaintext (b) (α, β)-core in ciphertext

Fig. 8. Comparison of (α, β)-core query times. (a) In plaintext. (b) In ciphertext.

degrees in different layers, thereby reducing computational overhead. Consequently, the index construction time of STE-BG* is substantially lower than that of other schemes.

For instance, in the DB dataset, the index construction times for BiCode-Index, SCS-Index, and STE-BG* are 0.473 s, 0.3 s, and 0.099 s, respectively. Similarly, in the ML dataset, the construction times for these schemes are 564.7 s, 480 s, and 4.62 s, respectively. These results highlight the significant advantage of STE-BG* in terms of index construction efficiency.

Secure Index Construction Time: The secure index construction times for PBG-Index and STE-BG are depicted in Fig. 6(b). It is evident that STE-BG requires more time to construct secure indexes compared to PBG-Index.

This difference arises because STE-BG is designed to support a wider range of query functionalities, necessitating the encryption of vertex degree information, node attributes, and edge weights. As a result, multiple index tables must be encrypted, leading to an increase in construction time. In contrast, PBG-Index only encrypts the vertex and edge tables, which reduces its index construction overhead.

In the ML dataset, the secure index construction times for PBG-Index and STE-BG are 14.5 s and 20.54 s, respectively. While STE-BG incurs a higher construction time due to its enriched query capabilities, it remains efficient given the security and functionality enhancements it provides.

D. Search Time

Fig. 7 presents the query efficiency for different (α, β) values, demonstrating that our scheme efficiently handles (α, β)-core, (α, β)-WC, and (α, β)-AWC queries.

Varying α: Fig. 7(a) and (c) illustrate the search time required for three query types in both plaintext and ciphertext settings as α varies while keeping β constant. The results reveal a clear trend: as α increases, query time decreases. This phenomenon arises because α serves as a degree constraint, meaning that as α grows, fewer vertices satisfy the condition. Consequently, the size of the adjacency matrix used to compute the (α, β)-core decreases, leading to improved computational efficiency.

Experimental results further demonstrate that (α, β)-core queries consistently exhibit lower query times compared to the other two query types. Additionally, (α, β)-AWC achieves faster query times than (α, β)-WC. The reason is that (α, β)-core solely considers vertex cohesion, ignoring edge weights, while (α, β)-WC extends (α, β)-core by searching for subgraphs that satisfy cohesion while maximizing edge weights. As for (α, β)-AWC, it applies attribute filtering to vertices first, thereby reducing the size of the adjacency matrix relative to (α, β)-WC, leading to enhanced query efficiency.

In the plaintext scenario, when $\alpha = 120$, the query times for (α, β)-core, (α, β)-WC, and (α, β) -AWC are 4.72 s, 5.42 s, and 5.12 s, respectively. Under the same conditions in the ciphertext scenario, the corresponding query times are 15.20 s, 21.67 s, and 18.19 s. Moreover, taking (α, β) -core queries as an example, the additional time overhead introduced by ciphertext queries compared to plaintext queries decreases from 4.57 s to 4.31 s as α increases from 60 to 300. Despite the increased ciphertext query time, the overall overhead remains within an acceptable range, ensuring data security in the encrypted environment.

Varying β: Fig. 7(b) and (d) illustrate the search time for the three query types in both plaintext and ciphertext settings as β varies while keeping α constant. Notably, the query time for all three types decreases as β increases, exhibiting a similar pattern to the variation in α. In the plaintext scenario, when $\alpha = 180$, the query times for (α, β)-core, (α, β)-WC, and (α, β)-AWC are 1.86 s, 2.06 s, and 1.98 s, respectively. Under the same conditions in the ciphertext scenario, the corresponding query times are 5.79 s, 6.48 s, and 6.12 s.

Taking (α, β)-WC queries as an example, in both plaintext and ciphertext environments, as α increases from 60 to 300, the additional query time introduced by ciphertext queries relative to plaintext queries ranges from 1.78 s to 4.57 s.

Comparison of (α, β)-core queries: Since most existing schemes focus on (α, β)-core queries, we compare their query efficiency with that of our proposed method. Fig. 8(a) presents the query time of various schemes for (α, β)-core queries in plaintext environments. The results show that as α and β increase, the query time decreases. The STE-BG* scheme exhibits similar query times to SCS-Index, whereas BiCode-Index takes longer. This is because BiCode-Index must traverse the entire graph for each (α, β)-core query. For instance, in dataset ML, when $\alpha = \beta = 120$, the query times for BiCode-Index, SCS-Index, and STE-BG* are 7.47 s, 4.17 s, and 4.615 s, respectively.

Fig. 8(b) shows (α, β)-core query times in ciphertext environments for different schemes. We observe that PBG-Index incurs longer query times compared to STE-BG. This is because PBG-Index must traverse both the node table and edge table, and as the graph size increases, the time required to read these tables grows accordingly. For dataset ML, when $\alpha = \beta = 120$, the query times for PBG-Index and STE-BG are 41.26 s and 15.20 s, respectively. Overall, our proposed scheme demonstrates a significant advantage in query efficiency.

X. Conclusion

In this paper, we introduced a structured encryption-based approach to enable richer query functionalities. First, we proposed a novel method for computing (α, β)-core by dynamically constructing adjacency matrices and performing row and column operations, ensuring accurate (α, β)-core query results. Second, to meet different user requirements, we proposed two additional community search methods: (α, β)-WC and (α, β)-AWC. Finally, experimental evaluations on real-world datasets confirmed the efficiency of our proposed scheme.

For future work, we plan to integrate end-to-end encryption and security protocols (e.g., TLS) to protect data in transit, along with multi-factor authentication and key exchange mechanisms to prevent unauthorized access. To counter advanced cryptanalytic attacks, such as differential and side-channel attacks, we aim to explore encryption algorithms resilient to such threats, incorporating randomization and other defensive mechanisms. Additionally, we will investigate integrity protection techniques based on hash functions and digital signatures to prevent data tampering.

Acknowledgment

The authors would like to thank Prof. Jianting Ning for helpful discussions and valuable suggestions on the technical aspects of this work. His insights contributed to improving the clarity and presentation of the manuscript.

References

[1] A. Beutel, W. Xu, V. Guruswami, C. Palow, and C. Faloutsos, "Copycatch: Stopping group attacks by spotting lockstep behavior in social networks," in *Proc. 22nd Int. Conf. World Wide Web*, 2013, pp. 119–130.

[2] N. Benchettara, R. Kanawati, and C. Rouveirol, "Supervised machine learning applied to link prediction in bipartite social networks," in *Proc. Int. Conf. Adv. Social Netw. Anal. Mining*, Odense, Denmark, 2010, pp. 326–330.

[3] Z. Liu, L. Cui, W. Guo, W. He, H. Li, and J. Gao, "Predicting hospital readmission using graph representation learning based on patient and disease bipartite graph," in *Proc. Database Syst. Adv. Appl. 25th Int. Conf.*, Jeju, South Korea, 2020, pp. 385–397.

[4] Y. Hao, M. Zhang, X. Wang, and C. Chen, "Cohesive subgraph detection in large bipartite networks," in *Proc. SSDBM 2020: 32nd Int. Conf. Sci. Stat. Database Manage.*, Vienna, Austria, 2020, pp. 22:1–22:4.

[5] L. Chen, C. Liu, R. Zhou, J. Xu, and J. Li, "Efficient exact algorithms for maximum balanced biclique search in bipartite graphs," in *Proc. Int. Conf. Manage. Data*, Virtual Event, China, 2021, pp. 248–260.

[6] K. Wang, X. Lin, L. Qin, W. Zhang, and Y. Zhang, "Efficient bitruss decomposition for large-scale bipartite graphs," in *Proc. 36th IEEE Int. Conf. Data Eng.*, Dallas, TX, USA, 2020, pp. 661–672.

[7] D. Ding, H. Li, Z. Huang, and N. Mamoulis, "Efficient fault-tolerant group recommendation using alpha-beta-core," in *Proc. ACM Conf. Inf. Knowl. Manage.*, Singapore, 2017, pp. 2047–2050.

[8] K. Wang, W. Zhang, X. Lin, Y. Zhang, L. Qin, and Y. Zhang, "Efficient and effective community search on large-scale bipartite graphs," in *Proc. 37th IEEE Int. Conf. Data Eng.*, Chania, Greece, 2021, pp. 85–96.

[9] Y. He, K. Wang, W. Zhang, X. Lin, and Y. Zhang, "Exploring cohesive subgraphs with vertex engagement and tie strength in bipartite graphs," *Inf. Sci.*, vol. 572, pp. 277–296, 2021.

[10] Y. Guan, R. Lu, Y. Zheng, S. Zhang, J. Shao, and G. Wei, "Achieving efficient and privacy-preserving (α, β)-core query over bipartite graphs in cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 1979–1993, May/Jun. 2023.

[11] R. Chen, B. Fung, P. Yu, and B. Desai, "Differentially private trajectory data publication," *VLDB J.*, vol. 23, no. 4, pp. 653–676, 2014.

[12] Z. Chang, L. Zou, and F. Li, "Privacy preserving subgraph matching on large graphs in cloud," in *Proc. Int. Conf. Manage. Data, SIGMOD Conf.*, San Francisco, CA, USA, 2016, pp. 199–213.

[13] Q. Liu, G. Wang, F. Li, S. Yang, and J. Wu, "Preserving privacy with probabilistic indistinguishability in weighted social networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 5, pp. 1417–1429, May 2017.

[14] L. Xu, J. Jiang, B. Choi, J. Xu, and S. S. Bhowmick, "Privacy preserving strong simulation queries on large graphs," in *Proc. 37th IEEE Int. Conf. Data Eng.*, Chania, Greece, 2021, pp. 1500–1511.

[15] T. Araki, J. Furukawa, K. Ohara, B. Pinkas, H. Rosemarin, and H. Tsuchida, "Secure graph analysis at scale," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Virtual Event, Republic of Korea, 2021, pp. 610–629.

[16] M. Chase and S. Kamara, "Structured encryption and controlled disclosure," in *Proc. Adv. Cryptol. 16th Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Singapore, 2010, pp. 577–594.

[17] N. Cao, Z. Yang, C. Wang, K. Ren, and W. Lou, "Privacy-preserving query over encrypted graph-structured data in cloud computing," in *Proc. Int. Conf. Distrib. Comput. Syst.*, Minneapolis, Minnesota, USA, 2011, pp. 393–402.

[18] G. S. Poh, M. S. Mohamad, and M. R. Z'aba, "Structured encryption for conceptual graphs," in *Proc. Adv. Inf. Comput. Secur. 7th Int. Workshop Secur.*, Fukuoka, Japan, 2012, pp. 105–122.

[19] C. Liu, L. Zhu, and J. Chen, "Graph encryption for top-k nearest keyword search queries on cloud," *IEEE Trans. Sustain. Comput.*, vol. 2, no. 4, pp. 371–381, Fourth Quarter 2017.

[20] M. Shen, B. Ma, L. Zhu, R. Mijumbi, X. Du, and J. Hu, "Cloud-based approximate constrained shortest distance queries over encrypted graphs with privacy protection," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 4, pp. 940–953, Apr. 2018.

[21] E. Ghosh, S. Kamara, and R. Tamassia, "Efficient graph encryption scheme for shortest path queries," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, Virtual Event, Hong Kong, 2021, pp. 516–525.

[22] M. Du, S. Wu, Q. Wang, D. Chen, P. Jiang, and A. Mohaisen, "GraphShield: Dynamic large graphs for secure queries with forward privacy," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 7, pp. 3295–3308, Jul. 2022.

[23] Y. Xue, L. Chen, Y. Mu, L. Zeng, F. Rezaeibagha, and R. H. Deng, "Structured encryption for knowledge graphs," *Inf. Sci.*, vol. 605, pp. 43–70, 2022.

[24] B. Liu, L. Yuan, X. Lin, L. Qin, W. Zhang, and J. Zhou, "Efficient (α, β)-core computation: An index-based approach," in *Proc. World Wide Web Conf.*, San Francisco, CA, USA, 2019, pp. 1130–1141.

[25] Q. Liu, X. Liao, X. Huang, J. Xu, and Y. Gao, "Distributed (α, β)-core decomposition over bipartite graphs," in *Proc. IEEE 39th Int. Conf. Data Eng.*, 2023, pp. 909–921.

[26] D. Li, X. Liang, R. Hu, L. Zeng, and X. Wang, "(α, β)-AWCS: (α, β)-attributed weighted community search on bipartite graphs," in *Proc. Int. Joint Conf. Neural Netw.*, Padua, Italy, 2022, pp. 1–8.

[27] Z. Xu et al., "Effective community search on large attributed bipartite graphs," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 37, no. 2, pp. 2359002:1–2359002:25, 2023.

[28] Q. Wang et al., "Privacy-preserving collaborative model learning: The case of word vector training," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 12, pp. 2381–2393, Dec. 2018.

[29] M. Li et al., "InstantCryptoGram: Secure image retrieval service," in *Proc. IEEE Conf. Comput. Commun.*, Honolulu, HI, USA, 2018, pp. 2222–2230.

[30] E. M. Kornaropoulos, N. Moyer, C. Papamanthou, and A. Psomas, "Leakage inversion: Towards quantifying privacy in searchable encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Los Angeles, CA, USA, 2022, pp. 1829–1842.

[31] S. Savvides, D. Khandelwal, and P. Eugster, "Efficient confidentiality-preserving data analytics over symmetrically encrypted datasets," in *Proc. VLDB Endowment*, vol. 13, no. 8, pp. 1290–1303, 2020.